



Python

Variables & Data Types (`int` , `float` , `str` , `bool`)

Variables are named containers in memory that store data values. Think of them as labeled boxes — you put a value inside the box and then refer to the box by its name anywhere in your code. This makes your programs reusable, readable, and easy to maintain.

Data Types are the category of data a variable can hold. Python automatically detects the data type when you assign a value (this is called **dynamic typing**). You don't need to declare the type in advance.

The four basic data types we are learning now:

- **int** (Integer): Whole numbers without any decimal point. Can be positive, negative, or zero.
- **float** (Floating-point number): Numbers that have a decimal point. Used when precision after the decimal is needed.
- **str** (String): Text or sequence of characters. Always enclosed in single quotes ' ' or double quotes " ".
- **bool** (Boolean): Logical values that can only be True or False. Used for yes/no or on/off decisions.

```
variable_name = value    # This is how you create and assign a variable
```

Rules for variable names:

- Must start with a letter (a-z, A-Z) or underscore _
- Can contain letters, numbers, and underscores only
- Cannot start with a number
- Case-sensitive (name and Name are different)
- Cannot use Python reserved keywords (if, for, True, False, etc.)

Checking data type:

```
type(variable_name)    # Returns the data type
```

Important Points:

- Python is dynamically typed — the same variable can hold different data types at different times.
- True and False must start with capital T and F.
- Strings can use single or double quotes (but be consistent in your code).
- Every value in Python is actually an object with a unique memory address.

Example 1: Creating variables of each data type

```
age = 25                # int
height = 5.9            # float
name = "Keval"          # str
is_student = True       # bool

print(age)              # 25
print(height)           # 5.9
print(name)             # Keval
print(is_student)       # True
```

Example 2: Using type() function

```
age = 25
height = 5.9
name = "Keval"
is_student = True

print(type(age))          # <class 'int'>
print(type(height))       # <class 'float'>
print(type(name))         # <class 'str'>
print(type(is_student))   # <class 'bool'>
```

Example 3: Reassigning variables (dynamic typing)

```
x = 10                    # initially int
print(x, type(x))        # 10 <class 'int'>

x = "Hello"              # now becomes str
print(x, type(x))        # Hello <class 'str'>
```

Example 4: Real-life variable usage

```
product_name = "Wireless Earbuds"
price = 1299.75
in_stock = True
quantity = 50

print("Product:", product_name)      # Product: Wireless Earbuds
print("Price: ₹", price)              # Price: ₹ 1299.75
print("In stock:", in_stock)          # In stock: True
print("Available quantity:", quantity) # Available quantity: 50
```

Example 5: Using variables in a small calculation

```
base_price = 1000          # int
gst_rate = 0.18            # float
product_name = "Laptop"    # str
is_discounted = False      # bool

final_price = base_price * (1 + gst_rate)

print("Product:", product_name)      # Product: Laptop
print("Final price after GST: ₹", final_price) # Final price after GST: ₹ 1180.0
print("Is discounted:", is_discounted) # Is discounted: False
```

Type Casting

Type Casting (also known as Type Conversion) is the process of converting one data type into another.

Python provides built-in functions to manually convert values between different types. This is necessary because Python is **strongly typed** — it does not automatically convert types in most operations. For example, you cannot directly add a string and a number without first converting them to the same type.

There are two types of type casting:

- **Explicit Type Casting:** Done manually by the programmer using functions like `int()`, `float()`, `str()`, `bool()`.
- **Implicit Type Casting:** Done automatically by Python in some cases (e.g., when mixing `int` and `float` in arithmetic).

Syntax & Key Points

```
int(value)      # Convert to integer
float(value)    # Convert to float
str(value)      # Convert to string
bool(value)     # Convert to boolean
```

Important Rules:

- `int()` removes the decimal part (truncates towards zero) when converting from `float`.
- `float()` can convert integers and valid numeric strings.
- `str()` can convert almost any value into its string representation.
- `bool()` follows truthy/falsy rules: most non-zero and non-empty values are `True`.
- Conversion fails with `ValueError` if the value cannot be converted (e.g., `int("abc")`).

Example 1: Basic type conversions

```
num_str = "25"          # string
num_int = int(num_str)  # string → int
print(num_int)          # 25
print(type(num_int))    # <class 'int'>

price = 99.99
price_int = int(price)  # float → int (truncates)
print(price_int)        # 99
```

Example 2: Converting to float and string

```
age = 25                # int
age_float = float(age)  # int → float
print(age_float)        # 25.0

score = 95
score_str = str(score)  # int → str
print(score_str)        # 95
print(type(score_str))  # <class 'str'>
```

Example 3: Handling user input with type casting

```
age_str = input("Enter your age: ")  # User enters: 25
age = int(age_str)                   # string → int

print("Your age is", age)             # Your age is 25
print("Next year you will be", age + 1) # Next year you will be 26
```

Example 4: Boolean conversion

```
print(bool(1))      # True
print(bool(0))      # False
print(bool(5.7))    # True
```

```
print(bool(""))          # False
print(bool("Python"))    # True
print(bool("False"))     # True    ← Common mistake
```

Example 5: Practical program combining multiple conversions

```
# Simple bill calculator
quantity_str = input("Enter quantity: ")    # e.g., 3
price_str = input("Enter price per item: ")  # e.g., 1299.75

quantity = int(quantity_str)                 # string → int
price = float(price_str)                     # string → float

total = quantity * price
total_str = str(total)                       # float → str for display

print("Total amount: ₹" + total_str)         # Total amount: ₹3899.25
print("Data type of total:", type(total))    # Data type of total: <class 'float'>
```

Edge Cases:

- `int(3.9)` → 3 (truncates, does not round)
- `int("3.14")` → `ValueError` (cannot convert float string directly to int)
- `float("inf")` → `inf` (special float value)
- `bool([])` or `bool({})` → `False` (empty containers are falsy)
- Very large numbers: Python handles them fine after casting.

Input/Output

Input/Output (I/O) refers to how a Python program interacts with the user or the outside world.

- **Output:** Displaying information from the program to the user or console. The main function used for output is `print()`.
- **Input:** Taking information from the user through the keyboard. The main function used for input is `input()`.

In simple terms:

- `print()` → Program speaks to the user (shows results, messages, etc.)
- `input()` → User speaks to the program (provides data)

Syntax & Key Points

Output - `print()`

```
print(value)                # Basic
print(value1, value2, ...)  # Multiple values
print(value, end='')        # Control ending character
print(value1, value2, sep=' ') # Control separator
```

Input - `input()`

```
variable = input()          # Without prompt
variable = input("Enter something: ") # With prompt message
```

Important Rules:

- `input()` **always returns a string** (str), even if user enters a number.

- `print()` by default adds a newline (`\n`) at the end.
- You can pass multiple arguments to `print()` separated by commas.
- `input()` waits for the user to press Enter before continuing.

Example 1: Basic Output using `print()`

```
print("Hello, World!")      # Hello, World!
print(100)                  # 100
print(5.75)                 # 5.75
print(True)                 # True
```

Example 2: Basic Input

```
name = input("Enter your name: ")    # User types: Keval
print("Hello,", name)                 # Hello, Keval
```

Example 3: Multiple values in `print()`

```
age = 25
city = "Ahmedabad"

print("My age is", age, "and I live in", city)
# My age is 25 and I live in Ahmedabad
```

Example 4: Controlling `print()` with `sep` and `end`

```
print("Python", "is", "awesome", sep="- ")
# Python-is-awesome

print("Hello", end=" ")
print("World!")
# Hello World!
```

Example 5: Input with type casting

```
age_str = input("Enter your age: ")    # User enters: 25
age = int(age_str)                     # Convert string to int

print("You are", age, "years old.")     # You are 25 years old.
print("Next year you will be", age + 1) # Next year you will be 26
```

Example 6: Mini Interactive Program

```
# Simple personalized greeting and calculation
name = input("What is your name? ")    # Keval
age_str = input("How old are you? ")   # 25

age = int(age_str)

print("Hello", name + "!")              # Hello Keval!
print("You are currently", age, "years old.") # You are currently 25 years old.
```

```
print("You will turn", age + 1, "next year.")      # You will turn 26 next year.

print("Thank you for using this program.", end=" ")
print("Have a great day!")
# Thank you for using this program. Have a great day!
```

Edge Cases:

- User presses Enter without typing anything → input() returns empty string ""
- Entering numbers with spaces → input() keeps them (use .strip() later)
- Very long input → Python can handle it, but good programs should validate
- Mixing print() with no arguments → prints just a blank line

Operators

Operators are special symbols that perform operations on values (operands). They allow you to do calculations, make comparisons, and combine conditions in Python.

Python has several types of operators. In this topic, we will cover the most commonly used ones without going too deep:

- **Arithmetic Operators:** Used for mathematical calculations (+, -, *, /, etc.)
- **Comparison Operators:** Used to compare two values (==, !=, >, <, etc.)
- **Logical Operators:** Used to combine multiple conditions (and, or, not)
- **Bitwise Operators:** Used for operations on binary numbers (advanced but basic understanding is enough)
- **Assignment Operators:** Used to assign values to variables (=, +=, -=, etc.)
- **Identity Operators:** Check if two objects are the same in memory (is, is not)
- **Membership Operators:** Check if a value exists in a sequence (in, not in)

Syntax & Key Points

```
# Arithmetic
a + b, a - b, a * b, a / b, a // b, a % b, a ** b

# Comparison
a == b, a != b, a > b, a < b, a >= b, a <= b

# Logical
condition1 and condition2, condition1 or condition2, not condition

# Bitwise (basic)
a & b, a | b, a ^ b, ~a, a << b, a >> b

# Assignment
x = 10, x += 5, x -= 3, x *= 2, x /= 4, x //= 2, x %= 3, x **= 2

# Identity
x is y, x is not y

# Membership
value in sequence, value not in sequence
```

Important Rules:

- Division / always returns float.
- Floor division // returns integer (rounds down).
- and returns True only if both are True.

- or returns True if at least one is True.
- is checks identity (same memory location), == checks value.
- in works with strings, lists, tuples, etc. (we will learn these later).

Example 1: Arithmetic Operators

```
a = 10
b = 3

print(a + b)      # 13
print(a - b)      # 7
print(a * b)      # 30
print(a / b)      # 3.3333333333333335
print(a // b)     # 3
print(a % b)      # 1
print(a ** b)     # 1000
```

Example 2: Comparison Operators

```
x = 5
y = 10

print(x == y)     # False
print(x != y)     # True
print(x > y)      # False
print(x < y)      # True
print(x >= 5)     # True
print(y <= 10)    # True
```

Example 3: Logical Operators

```
is_adult = True
has_id = False

print(is_adult and has_id)  # False
print(is_adult or has_id)   # True
print(not is_adult)         # False
```

Example 4: Assignment Operators

```
score = 50

score += 10      # score = score + 10
print(score)     # 60

score *= 2       # score = score * 2
print(score)     # 120

score //= 3      # score = score // 3
print(score)     # 40
```

Example 5: Identity and Membership Operators

```
a = 100
b = 100

print(a is b)           # True (for small integers)
print(a is not 200)     # True

name = "KevAl"
print("K" in name)       # True
print("xyz" not in name) # True
```

Example 6: Simple Eligibility Checker

```
age = 20
has_license = True
income = 50000

eligible = (age >= 18) and has_license and (income > 30000)

print("Eligible for loan?", eligible)  # Eligible for loan? True
```

Edge Cases:

- 10 / 3 gives float, not integer.
- 0 // 5 = 0, but 5 // 0 causes ZeroDivisionError.
- is works differently for large numbers vs small integers.
- Empty string "" in "abc" is True.

Control Flow (if / elif / else, Nested conditions, match-case)

Topic Explanation

Control Flow means controlling the order in which your code executes. By default, Python runs code line by line from top to bottom. Control flow statements let you make decisions and execute different blocks of code based on conditions.

- **if / elif / else:** The most fundamental way to make decisions in Python.
- **Nested conditions:** Putting one if-else inside another if-else.
- **match-case:** A modern, clean way to handle multiple conditions (introduced in Python 3.10+), similar to switch-case in other languages.

Why Control Flow is used:

- To make programs intelligent — they can behave differently based on user input or data.
- To handle different scenarios (e.g., login success/failure, grade calculation, age validation).

Syntax & Key Points

if / elif / else

```
if condition:
    # code block (must be indented)
    statement

elif another_condition:
    statement
```

```
else:
    statement
```

Nested if

```
if condition1:
    if condition2:
        # code
    else:
        # code
```

match-case (Python 3.10+)

```
match variable:
    case value1:
        # code
    case value2:
        # code
    case _ :           # default case (underscore)
        # code
```

Important Rules:

- Colon : is mandatory after if, elif, else, match, and case.
- Indentation (4 spaces) defines the code block.
- Conditions must evaluate to True or False.
- elif and else are optional.
- In match-case, _ acts as a wildcard (default) case.

Example 1: Simple if / else

```
age = 20

if age >= 18:
    print("You are eligible to vote.")           # You are eligible to vote.
else:
    print("You are not eligible to vote.")
```

Example 2: if / elif / else/`

```
marks = 85

if marks >= 90:
    print("Grade: A")                             # Grade: A (skipped)
elif marks >= 80:
    print("Grade: B")                             # Grade: B
elif marks >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

Example 3: Using match-case

```
day = "Monday"
```

```

match day:
    case "Monday":
        print("Start of the week")           # Start of the week
    case "Friday":
        print("End of the week")
    case _:
        print("Mid week")

```

Example 4: Nested if conditions

```

age = 25
has_license = True

if age >= 18:
    if has_license:
        print("You can drive.")           # You can drive.
    else:
        print("You need a license to drive.")
else:
    print("You are too young to drive.")

```

Example 5: Combined if-elif with logical operators

```

temperature = 35

if temperature > 40:
    print("Very Hot")
elif temperature >= 30 and temperature <= 40:
    print("Hot day")                       # Hot day
elif temperature >= 20:
    print("Pleasant")
else:
    print("Cold")

```

Example 6: Mini Program - Student Result System

```

marks = 78

if marks >= 90:
    grade = "A"
elif marks >= 80:
    grade = "B"
elif marks >= 70:
    grade = "C"
elif marks >= 60:
    grade = "D"
else:
    grade = "F"

print("Your grade is:", grade)             # Your grade is: C

# Nested check for distinction
if marks >= 75:
    if marks >= 90:
        print("Distinction with A grade")

```

```
else:
    print("First Division")           # First Division
```

Loops

Loops are used to repeat a block of code multiple times without writing the same code again and again. They help in performing repetitive tasks efficiently.

Python provides two main types of loops:

- **for loop:** Used when you know how many times you want to repeat (iteration over a sequence).
- **while loop:** Used when you want to repeat as long as a condition is True (condition-based).

Additional control statements:

- **break:** Immediately exits the loop.
- **continue:** Skips the current iteration and moves to the next one.
- **pass:** Does nothing (placeholder when a statement is required).

Nested loops: A loop inside another loop.

Syntax & Key Points

for loop

```
for variable in sequence:
    # code to repeat (indented)
```

while loop

```
while condition:
    # code to repeat (indented)
```

Control Statements

```
break        # Exit the loop completely
continue     # Skip current iteration
pass         # Do nothing (placeholder)
```

Nested Loops

```
for i in range(...):
    for j in range(...):
        # code
```

Important Rules:

- Indentation is mandatory to define the loop body.
- for loop works with any iterable (range, string, list, etc.).
- while loop must have a condition that eventually becomes False, otherwise it runs infinitely.
- range(start, stop, step) is commonly used with for loops.

Example 1: for loop with range()

```
for i in range(5):           # runs 0 to 4
    print(i)                 # 0
                             # 1
                             # 2
```

```
# 3
# 4
```

Example 2: while loop

```
count = 1
while count <= 5:
    print(count)      # 1 2 3 4 5 (each on new line)
    count += 1        # Important: update counter
```

Example 3: break, continue, pass

```
for i in range(10):
    if i == 3:
        continue      # skip 3
    if i == 7:
        break          # stop at 7
    print(i)           # 0 1 2 4 5 6
    pass              # does nothing
```

Example 4: for loop over string

```
name = "Keval"

for char in name:
    print(char)        # K
                     # e
                     # v
                     # a
                     # l
```

Example 5: while loop with user input

```
password = ""

while password != "python123":
    password = input("Enter password: ")
    if password == "python123":
        print("Access Granted")      # Access Granted (when correct)
    else:
        print("Wrong password, try again")
```

Example 6: Nested Loops - Printing Pattern

```
# Print a 3x5 rectangle of stars
for i in range(3):      # outer loop - rows
    for j in range(5):  # inner loop - columns
        print("*", end=" ") # * * * * *
    print()             # new line after each row
```

```
* * * * *
* * * * *
```

Lists

Lists are one of the most important and versatile built-in data structures in Python. A list is an **ordered, mutable (changeable), and indexed** collection of items. It can store items of different data types (int, float, str, bool, even other lists) in a single sequence.

Think of a list as a flexible shopping list or a dynamic array where you can add, remove, or modify items anytime.

Why Lists are used:

- To store and manage multiple related values together.
- To perform operations like sorting, searching, filtering, and modifying data easily.
- They are the go-to structure for most real-world data handling tasks.

Syntax & Key Points

```
my_list = [item1, item2, item3]      # Create list
empty_list = []                      # Empty list
mixed_list = [1, "Keval", 5.7, True] # Different data types allowed
```

Key Rules:

- Lists are **ordered** → items have positions (index starts at 0).
- Lists are **mutable** → you can change, add, or remove items after creation.
- Lists can be nested (list inside list).
- Negative indexing is supported (-1 = last item).
- Slicing: list[start:end:step] creates a new list.

Example 1: Creating and accessing lists

```
fruits = ["Apple", "Banana", "Cherry", "Mango"]

print(fruits)           # ['Apple', 'Banana', 'Cherry', 'Mango']
print(fruits[0])         # Apple
print(fruits[-1])        # Mango
print(fruits[1:3])       # ['Banana', 'Cherry']
```

Example 2: Modifying lists

```
numbers = [10, 20, 30]

numbers[1] = 25          # Change item
print(numbers)           # [10, 25, 30]

numbers.append(40)        # Add at end
print(numbers)           # [10, 25, 30, 40]
```

Example 3: Slicing and basic operations

```
scores = [85, 92, 78, 95, 88]

print(scores[::2])        # [85, 78, 88]    (every 2nd item)
print(scores[::-1])       # [88, 95, 78, 92, 85] (reverse)

print(len(scores))        # 5
```

```
print(max(scores))      # 95
print(min(scores))      # 78
```

Example 4: List with mixed types

```
student = ["Keval", 25, 5.9, True, ["Math", "Physics"]]

print(student[4])        # ['Math', 'Physics']
print(student[4][0])     # Math    (access nested list)
```

Example 5: Real-world list manipulation

```
cart = ["Laptop", "Mouse", "Keyboard"]

cart.insert(1, "Pendrive")    # Insert at position 1
print(cart)                  # ['Laptop', 'Pendrive', 'Mouse', 'Keyboard']

cart.remove("Mouse")          # Remove by value
print(cart)                  # ['Laptop', 'Pendrive', 'Keyboard']

popped = cart.pop()           # Remove and return last item
print(popped)                 # Keyboard
print(cart)                  # ['Laptop', 'Pendrive']
```

Here are ALL important list methods with syntax, use, and example:

```
# 1. Adding items
fruits.append("Orange")      # Adds at the end
# fruits → ['Apple', 'Banana', 'Cherry', 'Mango', 'Orange']

fruits.extend(["Pineapple", "Grapes"]) # Adds multiple items at the end
# fruits → ['Apple', ..., 'Orange', 'Pineapple', 'Grapes']

fruits.insert(2, "Kiwi")      # Insert at specific index
# fruits → ['Apple', 'Banana', 'Kiwi', 'Cherry', ...]

# 2. Removing items
fruits.remove("Banana")       # Remove first occurrence by value
# fruits → ['Apple', 'Kiwi', 'Cherry', ...]

popped = fruits.pop(3)        # Remove and return item at index (default: last)
print(popped)                 # Cherry

fruits.clear()                # Remove all items
# fruits → []

# 3. Searching & Counting
idx = fruits.index("Mango")    # Returns index of first occurrence
count = fruits.count("Apple")  # Count occurrences of value

# 4. Ordering
fruits.sort()                  # Sort in ascending order (in-place)
fruits.sort(reverse=True)      # Descending order

fruits.reverse()              # Reverse the list in-place
```

```
sorted_fruits = sorted(fruits)    # Returns new sorted list (does NOT modify original)

# 5. Copying
new_list = fruits.copy()          # Shallow copy (recommended)
new_list = list(fruits)           # Another way to copy

# 6. Other useful built-in functions
length = len(fruits)              # Number of items
total = sum([10, 20, 30])         # Sum of numbers
maximum = max([10, 20, 30])       # Maximum value
minimum = min([10, 20, 30])       # Minimum value
```

Edge Cases:

- Empty list: `len([]) == 0`, `pop()` on empty list → `IndexError`
- Duplicate items: `remove()` removes only first occurrence
- Nested lists: modifying inner list affects outer list
- Large lists: memory can become an issue (but Python handles millions of items well)

Tuples

Tuples are an ordered, **immutable** (unchangeable), and indexed collection of items. They are very similar to lists but once created, you cannot add, remove, or modify elements.

Think of a tuple as a **fixed, read-only list** — perfect when you want data that should never change during the program's execution.

Why Tuples are used:

- They are faster than lists (less memory overhead).
- They guarantee data integrity (no accidental changes).
- They can be used as keys in dictionaries (because they are immutable).
- Commonly used to return multiple values from functions.

Real-world use cases:

- Storing coordinates (latitude, longitude)
- Database records that should not be modified
- Function return values (e.g., status, message, data)
- Configuration constants (RGB colors, days of week)

Syntax & Key Points

```
my_tuple = (item1, item2, item3)    # With parentheses (recommended)
my_tuple = item1, item2, item3      # Without parentheses (tuple packing)
empty_tuple = ()                    # Empty tuple
single_item = (5,)                  # Single item tuple → note the comma
```

Key Rules:

- Tuples are **ordered** and **indexed** (index starts at 0, negative indexing supported).
- Tuples are **immutable** → no `append`, `remove`, `pop`, etc.
- Tuples can store mixed data types and can be nested.
- Slicing works exactly like lists and returns a new tuple.
- Tuples support **unpacking** — assigning values to multiple variables in one line.

Example 1: Creating and accessing tuples

```
coordinates = (10.5, 20.8)
print(coordinates)           # (10.5, 20.8)
print(coordinates[0])        # 10.5
print(coordinates[-1])       # 20.8
print(coordinates[1:])       # (20.8,)
```

Example 2: Tuple packing and unpacking

```
person = "Keval", 25, "Ahmedabad"    # packing
name, age, city = person               # unpacking

print(name)                           # Keval
print(age)                            # 25
print(city)                           # Ahmedabad
```

Example 3: Single item tuple

```
wrong = (5)                         # This is int, not tuple
correct = (5,)                      # This is tuple

print(type(wrong))                   # <class 'int'>
print(type(correct))                 # <class 'tuple'>
```

Example 4: Nested tuple and slicing

```
student = ("Keval", 25, ("Math", "Physics", "Chemistry"))

print(student[2])                    # ('Math', 'Physics', 'Chemistry')
print(student[2][1])                 # Physics
print(student[0:2])                  # ('Keval', 25)
```

Example 5: Tuple as function return

```
def get_user_info():
    return "Keval", 25, True          # returns tuple

name, age, is_student = get_user_info()

print(name, age, is_student)         # Keval 25 True
```

Example 6: Using tuple as dictionary key + multiple unpacking

```
# Tuple as dict key (possible because immutable)
location = (28.6139, 77.2090)        # Delhi coordinates
data = {location: "Delhi Capital"}

print(data[location])                # Delhi Capital

# Multiple unpacking with *
numbers = (1, 2, 3, 4, 5)
first, second, *rest = numbers

print(first)                         # 1
```

```
print(second)          # 2
print(rest)            # [3, 4, 5]
```

Tuples have very few methods because they are immutable. Here are ALL important ones:

```
numbers = (10, 20, 30, 20, 40)

# 1. Searching & Counting
idx = numbers.index(20)      # Returns first occurrence index → 1
count = numbers.count(20)    # Count occurrences → 2

# 2. Built-in functions (work on tuples)
length = len(numbers)        # 5
total = sum(numbers)         # 120
maximum = max(numbers)       # 40
minimum = min(numbers)       # 10

# 3. Other operations
sorted_tuple = sorted(numbers) # Returns new LIST (not tuple)
reversed_tuple = tuple(reversed(numbers)) # Returns new tuple

# 4. Concatenation & Repetition (creates new tuple)
tuple1 = (1, 2)
tuple2 = (3, 4)
combined = tuple1 + tuple2    # (1, 2, 3, 4)
repeated = tuple1 * 3         # (1, 2, 1, 2, 1, 2)

# 5. Membership test
print(20 in numbers)          # True
print(99 not in numbers)      # True
```

Edge Cases:

- Single item without comma → becomes int/float instead of tuple.
- Empty tuple: `len(()) == 0`
- Very large tuples are memory efficient compared to lists.
- Trying to modify: `TypeError`

Sets

Sets are an unordered, **mutable**, and **unique** (no duplicates) collection of items. They are highly optimized for membership testing (checking if an item exists) and mathematical set operations like union, intersection, etc.

Unlike lists and tuples, sets:

- Do **not** maintain order (items have no index).
- Automatically remove duplicates.
- Are extremely fast for checking if something exists.

Think of a set as a **bag of unique items** where order doesn't matter and you can't have two same items.

Why Sets are used:

- To remove duplicates from data quickly.
- To perform fast membership tests (in operator).
- To do mathematical set operations (union, intersection, etc.).
- To store unique values (tags, IDs, usernames).

Real-world use cases:

- Removing duplicate emails or phone numbers from a list.
- Checking if a user has already voted (unique IDs).
- Finding common friends between two people.
- Tag systems, keyword filtering, or permission checks.

Syntax & Key Points

```
my_set = {item1, item2, item3}      # Create set
empty_set = set()                   # Empty set ({} creates empty dict!)
mixed_set = {1, "Keval", 5.7, True}
```

Key Rules:

- Sets are **unordered** → no indexing or slicing (set[0] → TypeError).
- Sets are **mutable** → you can add/remove items.
- Sets **cannot contain mutable items** (no lists or dictionaries inside sets).
- Only **immutable** items (int, float, str, tuple, bool, frozenset) are allowed.
- Sets automatically remove duplicates.
- Very fast for in and not in checks.

Example 1: Creating sets and removing duplicates

```
numbers = {1, 2, 2, 3, 3, 3, 4}
print(numbers)                # {1, 2, 3, 4}

fruits = {"Apple", "Banana", "Apple", "Cherry"}
print(fruits)                  # {'Apple', 'Banana', 'Cherry'}    (order may vary)
```

Example 2: Membership testing (very fast)

```
students = {"Keval", "Rahul", "Priya"}

print("Keval" in students)     # True
print("Amit" in students)      # False
print("Rahul" not in students) # False
```

Example 3: Adding and removing items

```
tags = {"python", "coding"}

tags.add("django")             # Add single item
print(tags)                    # {'python', 'coding', 'django'}

tags.remove("coding")           # Remove item (raises error if not found)
print(tags)                     # {'python', 'django'}
```

Example 4: Mathematical set operations

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}
```

```

print(set_a.union(set_b))          # {1, 2, 3, 4, 5, 6}
print(set_a.intersection(set_b))  # {3, 4}
print(set_a.difference(set_b))    # {1, 2}
print(set_a.symmetric_difference(set_b)) # {1, 2, 5, 6}

```

Example 5: Converting other structures to set

```

numbers_list = [1, 2, 2, 3, 4, 4, 5]
unique = set(numbers_list)

print(unique)          # {1, 2, 3, 4, 5}
print(len(unique))     # 5

```

Example 6: Real-world unique user permissions

```

user_permissions = {"read", "write", "delete"}
admin_permissions = {"read", "write", "delete", "admin"}

# Check if user has all required permissions
required = {"read", "write"}
print(required.issubset(user_permissions))    # True

# Get combined permissions
all_perms = user_permissions.union(admin_permissions)
print(all_perms)                             # {'read', 'write', 'delete', 'admin'}

# Safe removal
user_permissions.discard("admin")              # No error even if not present
print(user_permissions)                       # {'read', 'write', 'delete'}

```

Here are ALL important set methods with syntax and example:

```

s = {1, 2, 3}

# 1. Adding items
s.add(4)          # Add single item
print(s)          # {1, 2, 3, 4}

s.update([5, 6, 7])  # Add multiple items from list/tuple
print(s)           # {1, 2, 3, 4, 5, 6, 7}

# 2. Removing items
s.remove(6)        # Remove item (raises KeyError if not found)
print(s)           # {1, 2, 3, 4, 5, 7}

s.discard(10)       # Remove item (NO error if not found)
print(s)           # {1, 2, 3, 4, 5, 7}

popped = s.pop()    # Remove and return random item
print(popped)       # e.g. 1 (random)

s.clear()           # Remove all items
print(s)           # set()

# 3. Set Operations (return new set)

```

```

set_a = {1, 2, 3}
set_b = {3, 4, 5}

union_set = set_a | set_b          # {1, 2, 3, 4, 5}
intersection_set = set_a & set_b    # {3}
difference_set = set_a - set_b      # {1, 2}
sym_diff = set_a ^ set_b            # {1, 2, 4, 5}

# 4. Comparison methods
print(set_a.issubset(set_b))        # False
print(set_a.isdisjoint(set_b))      # False
print(set_a.issuperset({1, 2}))     # True

# 5. Copy
new_set = s.copy()                  # Shallow copy

```

Edge Cases:

- Empty set: Must use `set()` not `{}`
- Adding mutable items (list) → `TypeError`
- `pop()` removes random element (unpredictable order)
- Very large sets are memory-efficient and extremely fast for lookup.

Dictionaries

Dictionaries (often called **dicts**) are Python's built-in data structure for storing **key-value pairs**. They are unordered (until Python 3.6 they were truly unordered; since Python 3.7 they maintain insertion order), mutable, and extremely fast for lookups.

Think of a dictionary as a real-life dictionary or a phone book: you look up a **key** (word/name) and instantly get its **value** (meaning/phone number). Keys must be unique and **immutable** (string, int, float, tuple, bool, etc.). Values can be anything (even another dictionary or list).

Why Dictionaries are used:

- To store and retrieve data by a meaningful key instead of index.
- For fast lookups, counting, configuration storage, JSON-like data, and mapping relationships.
- They are the backbone of almost every modern Python application.

Real-world use cases:

- User profiles (username → details)
- Configuration settings (key → value)
- Counting frequencies (word → count)
- Caching results
- API responses (JSON is converted to dict)

Syntax & Key Points

```

my_dict = {"key1": value1, "key2": value2}
empty_dict = {}                # or dict()
empty_dict = dict()            # recommended for clarity

```

Key Rules:

- Keys must be **immutable and hashable** (int, str, float, tuple, bool, None, frozenset).
- Values can be **any data type** (including lists, tuples, other dicts).
- Keys are **unique** — duplicate keys overwrite the previous value.

- Since Python 3.7, dictionaries maintain **insertion order**.
- Accessing a non-existent key raises `KeyError`.

Example 1: Creating and accessing dictionaries

```
student = {"name": "Keval", "age": 25, "city": "Ahmedabad"}

print(student)                # {'name': 'Keval', 'age': 25, 'city': 'Ahmedabad'}
print(student["name"])        # Keval
print(student.get("age"))     # 25
```

Example 2: Adding and updating values

```
student = {"name": "Keval"}

student["age"] = 25            # Add new key-value
student["city"] = "Ahmedabad"

student["age"] = 26           # Update existing key
print(student)               # {'name': 'Keval', 'age': 26, 'city': 'Ahmedabad'}
```

Example 3: Different key types

```
data = {
    101: "Roll Number",
    "is_student": True,
    (1, 2): "Tuple as key",      # Tuple is allowed
    None: "No value"
}

print(data[(1, 2)])             # Tuple as key
print(data[None])              # No value
```

Example 4: Nested dictionary

```
user = {
    "name": "Keval",
    "address": {
        "city": "Ahmedabad",
        "pincode": 380015
    },
    "skills": ["Python", "Django"]
}

print(user["address"]["city"])  # Ahmedabad
print(user["skills"][0])       # Python
```

Example 5: Looping through dictionary

```
scores = {"Math": 85, "Science": 92, "English": 78}

for key in scores:
    print(key, "→", scores[key])
```

```

# Math → 85
# Science → 92
# English → 78

for key, value in scores.items():
    print(key, value)
# Same output as above

```

Example 6: Real-world configuration + safe access

```

config = {
    "database": {"host": "localhost", "port": 5432},
    "debug": True,
    "version": "1.2.3"
}

# Safe access with .get()
host = config.get("database", {}).get("host")      # localhost
timeout = config.get("timeout", 30)                # default value if key missing

print(host)                                         # localhost
print(timeout)                                     # 30

# Updating nested dictionary safely
config.setdefault("logging", {})[ "level" ] = "INFO"
print(config["logging"])                           # {'level': 'INFO'}

```

Here are ALL important dictionary methods with syntax and example:

```

d = {"a": 1, "b": 2, "c": 3}

# 1. Adding / Updating
d["d"] = 4                # Add or update
d.update({"e": 5, "f": 6}) # Add multiple from another dict
d.setdefault("g", 100)    # Add only if key doesn't exist

# 2. Removing
value = d.pop("b")        # Remove and return value → 2
del d["c"]                # Delete key
d.clear()                 # Remove all items

# 3. Retrieving
keys = d.keys()            # dict_keys(['a', 'd', ...])
values = d.values()        # dict_values([1, 4, ...])
items = d.items()          # dict_items([('a', 1), ...])

# 4. Safe access
val = d.get("missing", "default") # Returns default instead of KeyError

# 5. Copying
new_d = d.copy()           # Shallow copy
new_d = dict(d)            # Another way

# 6. Other useful methods
print("a" in d)            # True (checks keys only)
print(len(d))              # Number of key-value pairs

```

```
# 7. Python 3.9+ operators (very useful)
dict1 = {"a": 1, "b": 2}
dict2 = {"b": 3, "c": 4}

merged = dict1 | dict2          # {'a': 1, 'b': 3, 'c': 4} (dict2 wins on conflict)
dict1 |= dict2                  # In-place merge
```

Edge Cases:

- Modifying dictionary during iteration → `RuntimeError` (use `list(d.keys())` or `d.copy()`).
- Using mutable keys (list) → `TypeError: unhashable type`.
- Very large dictionaries are memory-efficient and extremely fast.
- `d.get(key)` returns `None` by default if key missing.

List Comprehensions

List Comprehension is a concise, elegant, and Pythonic way to create a new list from an existing iterable (like list, tuple, range, string, etc.) in a single line.

It combines **iteration**, **condition**, and **expression** into one readable expression. Instead of writing a traditional for loop with `append()`, you can create and transform lists much more cleanly.

Why List Comprehensions are used:

- They make code shorter, more readable, and more expressive.
- They are faster than traditional loops in most cases.
- They are widely used in data processing, filtering, and transformation tasks.

Real-world use cases:

- Extracting specific data from a list of records
- Filtering even/odd numbers or valid emails
- Transforming data (squaring numbers, converting strings to uppercase)
- Creating lists of calculated values quickly

Syntax & Key Points

Basic Syntax:

```
new_list = [expression for item in iterable]
```

With Condition (Filtering):

```
new_list = [expression for item in iterable if condition]
```

With if-else (Conditional Expression):

```
new_list=[expression_if_true if condition else expression_if_false for item in iterable]
```

Important Rules:

- The `for` part is mandatory.
- The `if` condition (filtering) comes after the `for`.
- For if-else, the condition comes before the `for`.
- You can nest multiple for loops (for Cartesian product or flattening).
- List comprehensions are evaluated from left to right.

Example 1: Simple list creation

```
numbers = [x for x in range(1, 6)]
print(numbers)           # [1, 2, 3, 4, 5]

squares = [x**2 for x in range(1, 6)]
print(squares)           # [1, 4, 9, 16, 25]
```

Example 2: With condition (filtering)

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

even_numbers = [x for x in numbers if x % 2 == 0]
print(even_numbers)      # [2, 4, 6, 8, 10]

positive = [x for x in [-5, 3, -2, 8, -1] if x > 0]
print(positive)           # [3, 8]
```

Example 3: if-else in list comprehension

```
numbers = [1, 2, 3, 4, 5]

result = ["Even" if x % 2 == 0 else "Odd" for x in numbers]
print(result)             # ['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

Example 4: Working with strings

```
names = ["keval", "rahul", "priya", "amit"]

capitalized = [name.capitalize() for name in names]
print(capitalized)        # ['Keval', 'Rahul', 'Priya', 'Amit']

long_names = [name for name in names if len(name) > 4]
print(long_names)         # ['keval', 'priya']
```

Example 5: Nested loops in list comprehension

```
# Create all pairs from two lists
colors = ["Red", "Blue"]
sizes = ["S", "M", "L"]

combinations = [color + "-" + size for color in colors for size in sizes]
print(combinations)
# ['Red-S', 'Red-M', 'Red-L', 'Blue-S', 'Blue-M', 'Blue-L']

# Flatten a nested list
nested = [[1, 2], [3, 4], [5, 6]]
flattened = [num for sublist in nested for num in sublist]
print(flattened)          # [1, 2, 3, 4, 5, 6]
```

Example 6: Real-world data transformation

```

students = [
    {"name": "Keval", "marks": 85},
    {"name": "Rahul", "marks": 72},
    {"name": "Priya", "marks": 95}
]

# Get names of students who scored above 80
top_students = [student["name"] for student in students if student["marks"] > 80]
print(top_students)          # ['Keval', 'Priya']

# Create list of (name, grade) tuples
grades = [(s["name"], "Pass" if s["marks"] >= 60 else "Fail") for s in students]
print(grades)                # [('Keval', 'Pass'), ('Rahul', 'Pass'), ('Priya', 'Pass')]

```

All Methods / Variations

- Basic: [expression for item in iterable]
- With filter: [expression for item in iterable if condition]
- With if-else: [true_expr if condition else false_expr for item in iterable]
- Nested for: [expr for x in list1 for y in list2]
- With multiple conditions: [expr for item in iterable if cond1 and cond2]
- Using functions inside: [len(word) for word in words]

Edge Cases:

- Empty iterable → returns empty list.
- Very complex expressions inside comprehension → hurts readability (avoid).
- Memory usage: Creating large lists in one go can consume significant memory.
- Side effects inside comprehension (like function calls with prints) → generally not recommended.

Dictionary Comprehensions

Dictionary Comprehension is a concise and Pythonic way to create a new dictionary from an iterable in a single line, similar to list comprehension.

It allows you to build key-value pairs dynamically using an expression for both the key and the value, along with optional filtering conditions.

This feature was introduced in Python 2.7 and is extremely useful for transforming data, creating mappings, or processing collections quickly and cleanly.

Why Dictionary Comprehensions are used:

- To create dictionaries in a readable, compact way.
- To transform one data structure into a dictionary efficiently.
- To filter data while building key-value pairs.
- They are faster and more elegant than traditional for loops with manual dictionary updates.

Real-world use cases:

- Creating a word-to-length mapping from a list of words.
- Converting a list of tuples into a dictionary.
- Building frequency counters.
- Transforming configuration data or API responses.
- Creating lookup tables (e.g., student ID → name).

Syntax & Key Points

Basic Syntax:

```
new_dict = {key_expression: value_expression for item in iterable}
```

With Condition (Filtering):

```
new_dict = {key_expression: value_expression for item in iterable if condition}
```

Important Rules:

- The for part is mandatory.
- Key expression must produce a **hashable** (immutable) value.
- You can use if to filter items (placed at the end).
- Dictionary comprehensions always create a **new** dictionary.
- Since Python 3.7, the resulting dictionary maintains insertion order.
- You cannot directly use if-else inside the value part like in list comprehensions (use ternary if needed).

Example 1: Simple dictionary creation

```
squares = {x: x**2 for x in range(1, 6)}  
print(squares)                # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}  
  
numbers = {x: "even" if x % 2 == 0 else "odd" for x in range(1, 6)}  
print(numbers)                # {1: 'odd', 2: 'even', 3: 'odd', 4: 'even', 5: 'odd'}
```

Example 2: From existing list

```
fruits = ["apple", "banana", "cherry"]  
  
fruit_lengths = {fruit: len(fruit) for fruit in fruits}  
print(fruit_lengths)          # {'apple': 5, 'banana': 6, 'cherry': 6}
```

Example 3: With filtering condition

```
prices = {"apple": 120, "banana": 60, "cherry": 200, "mango": 90}  
  
affordable = {item: price for item, price in prices.items() if price <= 100}  
print(affordable)             # {'banana': 60, 'mango': 90}
```

Example 4: Converting list of tuples

```
students = [("Keval", 85), ("Rahul", 72), ("Priya", 95)]  
  
score_dict = {name: marks for name, marks in students}  
print(score_dict)             # {'Keval': 85, 'Rahul': 72, 'Priya': 95}
```

Example 5: Real-world data transformation

```
# Student data with filtering and transformation  
raw_data = [  
    {"id": 101, "name": "Keval", "marks": 85, "active": True},
```

```

        {"id": 102, "name": "Rahul", "marks": 45, "active": True},
        {"id": 103, "name": "Priya", "marks": 92, "active": False}
    ]

    # Create dict of active students with grade
    student_grades = {
        student["id"]: {
            "name": student["name"],
            "grade": "Pass" if student["marks"] >= 60 else "Fail"
        }
        for student in raw_data
        if student["active"] and student["marks"] >= 50
    }

    print(student_grades)
    # {101: {'name': 'Keval', 'grade': 'Pass'}}

```

Example 6: Swapping keys and values

```

original = {"a": 1, "b": 2, "c": 3}

swapped = {value: key for key, value in original.items()}
print(swapped)                # {1: 'a', 2: 'b', 3: 'c'}

```

All Methods / Variations

- Basic: {key_expr: value_expr for item in iterable}
- With filter: {key_expr: value_expr for item in iterable if condition}
- Using .items(): {new_key: new_value for key, value in old_dict.items()}
- With ternary in value: {key: true_value if condition else false_value for key in iterable}
- Nested: {outer: {inner_key: inner_value for inner in inner_list} for outer in outer_list}
- From zip(): {k: v for k, v in zip(keys_list, values_list)}

Edge Cases:

- If two items produce the same key, the last one overwrites previous values.
- Using non-hashable keys (lists, sets, dicts) → TypeError.
- Empty iterable → returns empty dictionary {}.
- Large data: Dictionary comprehensions are memory-efficient but still create the full dict in memory.

Set Comprehensions

Set Comprehension is a concise and Pythonic way to create a new **set** from an iterable in a single line, similar to list and dictionary comprehensions.

It automatically removes duplicates and produces an unordered collection of unique items. Set comprehensions are particularly useful when you want to create a set while applying transformations or filtering conditions at the same time.

Why Set Comprehensions are used:

- To quickly create sets with unique elements.
- To combine filtering and transformation in one readable expression.
- To perform fast membership testing after creating unique transformed data.
- They are more compact and often faster than traditional loops with set.add().

Real-world use cases:

- Removing duplicates while transforming data (e.g., unique uppercase words).
- Creating sets of unique IDs, tags, or categories.
- Extracting unique characters from text.
- Building unique domain names or email domains from a list.

Syntax & Key Points

Basic Syntax:

```
new_set = {expression for item in iterable}
```

With Condition (Filtering):

```
new_set = {expression for item in iterable if condition}
```

Important Rules:

- The result is always a **set** → unordered and no duplicates.
- The for part is mandatory.
- The if condition (for filtering) comes at the end.
- Expression must produce **hashable** values (since sets can only contain hashable items).
- Set comprehensions automatically eliminate duplicate values.
- No direct support for if-else like in list comprehensions (use ternary if needed).

Example 1: Simple set creation

```
numbers = {x for x in range(1, 11)}
print(numbers)                # {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

squares = {x**2 for x in range(1, 6)}
print(squares)                # {1, 4, 9, 16, 25}
```

Example 2: Removing duplicates with transformation

```
words = ["apple", "banana", "Apple", "cherry", "banana"]

unique_upper = {word.upper() for word in words}
print(unique_upper)          # {'APPLE', 'BANANA', 'CHERRY'}
```

Example 3: With filtering condition

```
numbers = [10, 15, 20, 25, 30, 35, 40]

even_squares = {x**2 for x in numbers if x % 2 == 0}
print(even_squares)          # {400, 100, 900}    (order may vary)
```

Example 4: Unique characters from string

```
sentence = "Python is awesome"

unique_chars = {char.lower() for char in sentence if char.isalpha()}
```

```
print(unique_chars)
# {'p', 'y', 't', 'h', 'o', 'n', 'i', 's', 'a', 'w', 'e', 'm'}
```

Example 5: Real-world unique data processing

```
users = [
    {"id": 101, "tags": ["python", "django", "web"]},
    {"id": 102, "tags": ["python", "flask", "web"]},
    {"id": 103, "tags": ["java", "spring"]}
]

# Collect all unique tags across all users
all_tags = {tag for user in users for tag in user["tags"]}
print(all_tags)
# {'python', 'django', 'web', 'flask', 'java', 'spring'}

# Unique tags with length > 4
long_tags = {tag for user in users for tag in user["tags"] if len(tag) > 4}
print(long_tags)
# {'python', 'django', 'flask', 'spring'}
```

Example 6: Using ternary inside set comprehension

```
numbers = range(-5, 6)

pos_neg = {"positive" if x > 0 else "negative" if x < 0 else "zero" for x in numbers}
print(pos_neg)
# {'positive', 'negative', 'zero'}
```

All Methods / Variations

- Basic: {expression for item in iterable}
- With filter: {expression for item in iterable if condition}
- Nested for loops: {expr for outer in outer_list for inner in outer}
- With ternary: {true_expr if condition else false_expr for item in iterable}
- From other structures: {item.upper() for item in my_list}
- Combining with functions: {len(word) for word in words if word}

Edge Cases:

- Empty iterable → returns empty set set()
- All items produce same value → results in a set with only one element
- Using non-hashable expression result (e.g., list) → TypeError: unhashable type
- Very large sets can consume significant memory

Nested Data Structures

Nested Data Structures refer to placing one data structure inside another. This allows you to represent complex, real-world data in a hierarchical way.

Common combinations include:

- List inside List
- Dictionary inside List

- List inside Dictionary
- Dictionary inside Dictionary
- Tuple inside List / Dictionary, etc.

This is one of the most powerful features in Python because it lets you model structured data like JSON, database records, configuration files, and application data naturally.

Why Nested Data Structures are used:

- To represent hierarchical or relational data (e.g., students with multiple subjects).
- To handle complex data received from APIs, JSON files, or databases.
- To organize related information together in a logical structure.

Real-world use cases:

- Student records (list of dictionaries)
- E-commerce cart (dictionary containing list of products)
- JSON API responses
- Game data (player stats with inventory list)
- Configuration with nested sections

Syntax & Key Points

```
# List of Dictionaries
students = [
    {"name": "Keval", "age": 25, "marks": {"math": 85, "science": 92}},
    {"name": "Priya", "age": 23, "marks": {"math": 95, "science": 88}}
]

# Dictionary containing Lists
company = {
    "name": "TechCorp",
    "employees": ["Keval", "Rahul", "Priya"],
    "departments": {
        "engineering": ["Alice", "Bob"],
        "hr": ["Sara"]
    }
}
```

Key Rules:

- You can nest any data structure inside another (list, tuple, set, dict).
- Access is done using multiple indexing or key lookups (chaining [] or .get()).
- Mutability depends on the outer and inner structures.
- Be careful with deep nesting — it can make code harder to read.

Example 1: List of Dictionaries

```
students = [
    {"name": "Keval", "age": 25},
    {"name": "Priya", "age": 23}
]

print(students[0]["name"])      # Keval
print(students[1]["age"])      # 23
```

Example 2: Dictionary containing List

```
fruits = {
    "summer": ["Mango", "Watermelon"],
    "winter": ["Orange", "Apple"]
}

print(fruits["summer"])          # ['Mango', 'Watermelon']
print(fruits["summer"][0])       # Mango
```

Example 3: Dictionary inside Dictionary

```
user_profile = {
    "personal": {
        "name": "Keval",
        "city": "Ahmedabad"
    },
    "education": {
        "degree": "B.Tech",
        "year": 2023
    }
}

print(user_profile["personal"]["city"])    # Ahmedabad
print(user_profile["education"]["degree"]) # B.Tech
```

Example 4: Nested List (Matrix)

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matrix[1][2])          # 6 (row 1, column 2)
```

Example 5: Complex Real-world Nested Structure

```
ecommerce_order = {
    "order_id": "ORD12345",
    "customer": {
        "name": "Keval",
        "address": {
            "city": "Ahmedabad",
            "pincode": 380015
        }
    },
    "items": [
        {"product": "Laptop", "qty": 1, "price": 45000},
        {"product": "Mouse", "qty": 2, "price": 800}
    ],
    "status": "Delivered"
}
```

```
# Accessing deeply nested data
print(ecommerce_order["customer"]["address"]["city"])    # Ahmedabad

# Looping through nested list
total = 0
for item in ecommerce_order["items"]:
    total += item["qty"] * item["price"]

print("Total amount:", total)                            # Total amount: 46600
```

Edge Cases:

- Deep nesting (5+ levels) makes debugging difficult.
- Modifying inner mutable structures affects the outer one.
- Accessing non-existent keys/indexes raises `KeyError` or `IndexError`.
- Mixing mutable and immutable nested types changes behavior.

Internal Understanding – Mutability vs Immutability, Time Complexity Basics & Memory References

A. Mutability vs Immutability

Mutability means an object's value can be changed after it is created without changing its identity (memory address).

Immutability means once an object is created, its value cannot be changed. Any "modification" creates a completely new object.

- **Mutable Types:** list, dict, set
- **Immutable Types:** int, float, str, bool, tuple, frozenset, None

B. Time Complexity Basics

Time Complexity describes how the runtime of an operation grows as the input size increases. It helps us choose the right data structure and algorithm.

Common notations (Big O):

- **O(1)** – Constant time (very fast, doesn't depend on size)
- **O(n)** – Linear time
- **O(log n)** – Logarithmic time
- **O(n²)** – Quadratic time

C. Memory References (Identity)

Every object in Python has a unique **memory address** (id).

- `id(object)` returns this address.
- Variables are just **references** (pointers) to objects in memory.
- Understanding references is crucial to avoid bugs with mutable objects.

Why these concepts matter together:

Mutability affects how memory references behave. Understanding both helps you write efficient, bug-free code and choose correct data structures.

Real-world importance:

- Preventing accidental data modification bugs
- Optimizing performance in large applications
- Debugging reference-related issues
- Choosing between list vs tuple, dict vs other structures

Syntax & Key Points

Mutability Check:

```
id(object)          # Returns memory address
```

Common Mutable Operations (change in-place):

- list.append(), list[0] = x
- dict["key"] = value
- set.add()

Immutable Operations (create new object):

- x = x + 1 (for int)
- s = s.upper() (for string)
- t = t + (4,) (for tuple)

Time Complexity (Basic Operations):

- List: append → O(1) amortized, indexing → O(1), search → O(n)
- Dict/Set: lookup (in), insert, delete → O(1) average
- Tuple: indexing → O(1), but cannot modify

Example 1: Mutability Demonstration

```
# Mutable - List
a = [1, 2, 3]
b = a          # b references the same list as a
b.append(4)

print(a)       # [1, 2, 3, 4]
print(b)       # [1, 2, 3, 4]
print(id(a) == id(b))  # True (same object)
```

Example 2: Immutability Demonstration

```
# Immutable - String
x = "hello"
y = x
y = y.upper()  # Creates NEW string

print(x)       # hello      (original unchanged)
print(y)       # HELLO
print(id(x) == id(y))  # False (different objects)
```

Example 3: Memory References with Numbers

```
a = 100
b = 100

print(id(a) == id(b))  # True (Python caches small integers)
```

Example 4: Mutable vs Immutable in Functions

```
def modify_list(lst):
    lst.append(99)          # Modifies original list

def modify_number(num):
    num += 10              # Creates new number, doesn't affect original

my_list = [1, 2]
modify_list(my_list)
print(my_list)             # [1, 2, 99]    ← Changed!

my_num = 5
modify_number(my_num)
print(my_num)             # 5             ← Unchanged!
```

Example 5: Time Complexity in Action

```
# Fast operations (O(1))
my_dict = {"name": "Keval"}
print(my_dict["name"])     # Very fast lookup

my_set = {1, 2, 3}
print(2 in my_set)         # Very fast membership test

# Slower operation (O(n))
my_list = [1, 2, 3, 4, 5]
print(3 in my_list)        # Scans the entire list in worst case
```

Example 6: Combined Real-world Scenario

```
# Configuration with nested mutable structure
config = {
    "user": {"name": "Keval", "settings": [True, False]},
    "version": "1.0"
}

# Dangerous: Shallow reference
settings_ref = config["user"]["settings"]
settings_ref.append("dark_mode")    # Modifies original config!

print(config["user"]["settings"])
# ['True', 'False', 'dark_mode']    ← Original changed!

# Safe way - create copy
import copy
safe_settings = copy.deepcopy(config["user"]["settings"])
```

All Important Points / Variations

Mutability Summary:

- **Mutable:** list, dict, set, bytearray
- **Immutable:** int, float, str, tuple, frozenset, bool, None, complex

Time Complexity Quick Reference:

- **Dictionary & Set:** Average $O(1)$ for lookup, insert, delete
- **List:** $O(1)$ for append and index access, $O(n)$ for search and insert in middle

- **Tuple:** $O(1)$ for index access (faster than list)

Memory Reference Rules:

- Assignment `b = a` makes `b` point to the same object as `a`
- For immutable types, operations create new objects
- For mutable types, in-place methods modify the existing object

Edge Cases:

- Small integers (-5 to 256) are cached by Python → same id
- String interning: Some strings may share memory
- Modifying mutable object inside tuple is still possible (tuple only protects its own structure)
- Shallow copy vs Deep copy is critical with nested structures

Strings

Strings (`str`) are immutable sequences of Unicode characters. They are one of the most frequently used data types in Python.

A string is **immutable**, meaning once created, its content cannot be changed. Any modification (like uppercasing or replacing) creates a **new string** object.

Strings support indexing, slicing, iteration, and a rich set of built-in methods for manipulation, searching, formatting, and validation.

Why Strings are used:

- To handle text data (user input, file content, API responses, logs, etc.).
- For data processing, cleaning, and transformation.
- As keys in dictionaries (because they are immutable and hashable).

Real-world use cases:

- Processing user names, emails, and passwords
- Parsing log files and CSV data
- Generating reports and formatted output
- Web scraping and text analysis
- Regular expression-based pattern matching

Syntax & Key Points

```
my_str = "Hello World"           # Double quotes
my_str = 'Hello World'           # Single quotes
my_str = """Multi-line
string"""                        # Triple quotes for multi-line
```

Important Rules:

- Strings are **immutable** → cannot do `s[0] = 'X'`
- Strings are **ordered and indexed** (0-based indexing, negative indexing supported)
- Strings support **slicing**: `string[start:end:step]`
- Strings are **iterable** (you can loop over characters)
- Case-sensitive by default

Example 1: Indexing & Slicing

```
s = "Python Programming"

print(s[0])          # P
print(s[-1])         # g
print(s[0:6])        # Python
print(s[7:])         # Programming
print(s[::2])        # Pto rgamn
print(s[::-1])       # gnimmargorP nohtyP (reverse)
```

Example 2: Basic String Methods

```
text = "  Hello Python  "

print(text.upper())    #  HELLO PYTHON
print(text.lower())    #  hello python
print(text.strip())    # Hello Python
print(text.replace("Python", "World")) #  Hello World
```

Example 3: Searching and Checking

```
sentence = "Python is awesome and Python is easy"

print(sentence.find("Python"))    # 0
print(sentence.rfind("Python"))   # 21
print(sentence.count("Python"))   # 2

print("awesome" in sentence)      # True
print(sentence.startswith("Python"))# True
print(sentence.endswith("easy"))  # True
```

Example 4: Splitting and Joining

```
data = "Keval,25,Ahmedabad,Python"

parts = data.split(",")          # ['Keval', '25', 'Ahmedabad', 'Python']
print(parts)

joined = " | ".join(parts)
print(joined)                    # Keval | 25 | Ahmedabad | Python
```

Example 5: Real-world Text Processing

```
raw_input = "  User123   ,   keval@example.com   ,   Active   "

# Clean and process
cleaned = [field.strip().lower() for field in raw_input.split(",")]

username = cleaned[0]
email = cleaned[1]
status = cleaned[2].capitalize()

print(f"Username: {username}")    # Username: user123
```

```
print(f"Email: {email}")          # Email: keval@example.com
print(f"Status: {status}")        # Status: Active
```

Here is a comprehensive list of most important string methods:

```
s = "Python Programming"

# Case Conversion
s.upper()          # PYTHON PROGRAMMING
s.lower()          # python programming
s.title()          # Python Programming
s.capitalize()     # Python programming
s.swapcase()       # pYTHON pROGRAMMING

# Stripping Whitespace
s.strip()          # Remove from both sides
s.lstrip()         # Remove from left
s.rstrip()         # Remove from right

# Searching & Counting
s.find("Python")   # Returns index or -1
s.rfind("Python")  # Search from right
s.index("Python")  # Like find but raises ValueError
s.count("o")       # Count occurrences

# Checking
s.startswith("Py")
s.endswith("ing")
s.isalpha()        # Only letters?
s.isdigit()        # Only digits?
s.isalnum()        # Letters + digits?
s.isspace()        # Only whitespace?

# Replacing & Splitting
s.replace("Python", "Java")
s.split()          # Split on whitespace (default)
s.split(",")       # Split on delimiter
s.splitlines()     # Split on line breaks

# Joining
"-".join(["Python", "is", "fun"]) # Python-is-fun

# Formatting (old style)
s.center(30, "*")  # Center align
s.ljust(30)        # Left justify
s.rjust(30)        # Right justify
```

String Formatting

A. f-strings (Recommended - Python 3.6+)

```
name = "Keval"
age = 25
city = "Ahmedabad"

print(f"My name is {name}, I am {age} years old and live in {city}.")
# My name is Keval, I am 25 years old and live in Ahmedabad.
```

```
# With expressions
print(f"Next year I will be {age + 1}")          # Next year I will be 26

# Formatting numbers
pi = 3.14159
print(f"Pi is approximately {pi:.2f}")          # Pi is approximately 3.14
print(f"Score: {95:03d}")                        # Score: 095
```

B. .format() method

```
print("My name is {}, I am {} years old.".format("Keval", 25))

# With named placeholders
print("Hello {name}, welcome to {city}".format(name="Keval", city="Ahmedabad"))
```

Regex (re Module) - Basic Introduction

```
import re

text = "My phone number is 98765-43210 and email is keval@example.com"

# Basic patterns
print(re.search(r"\d{5}-\d{4}", text))          # Finds phone number
print(re.findall(r"\w+@\w+\.\w+", text))        # Finds email

# Common functions
re.search(pattern, text)                        # Returns first match
re.findall(pattern, text)                       # Returns all matches as list
re.sub(pattern, replacement, text)              # Replace matches
```

Common Regex Patterns:

- \d → digit
- \w → word character (letter, digit, underscore)
- \s → whitespace
- + → one or more
- → zero or more
- ? → optional

Example:

```
email_pattern = r"[\w\.-]+@[\w\.-]+\.\w+"
emails = re.findall(email_pattern, "Contact: keval@email.com, support@test.org")
print(emails)  # ['keval@email.com', 'support@test.org']
```

Functions

A **function** is a reusable block of code that performs a specific task. Functions help in organizing code, avoiding repetition, and making programs modular and easier to debug.

Functions follow the **DRY** principle (Don't Repeat Yourself). Once defined, you can call them multiple times with different inputs.

Why Functions are used:

- To break down complex problems into smaller, manageable parts
- To reuse code without rewriting it
- To improve readability and maintainability

Real-world use cases:

- Calculating bill totals, validating user input, processing data, etc.

Syntax & Key Points

```
def function_name(parameters):  
    """Docstring - description of the function"""  
    # function body  
    return value    # optional
```

Rules:

- Function name should be lowercase with underscores (snake_case)
- def keyword is used to define a function
- Colon : is mandatory after parameters
- Indentation defines the function body
- return statement is optional (if omitted, function returns None)

Basic Example

```
def greet():  
    print("Hello, Welcome to Python!")    # Hello, Welcome to Python!  
  
greet()                                # Calling the function
```

Intermediate Example

```
def add_two_numbers(a, b):  
    result = a + b  
    return result  
  
sum_value = add_two_numbers(10, 25)  
print(sum_value)                    # 35
```

Advanced Example

```
def calculate_area(length, width):  
    """Calculate area of rectangle"""  
    if length <= 0 or width <= 0:  
        return "Invalid dimensions"  
    return length * width  
  
print(calculate_area(5, 8))          # 40  
print(calculate_area(-3, 4))         # Invalid dimensions
```

Parameters & Return Values

Parameters are variables listed inside the function definition. They receive values when the function is called (**arguments**).

Return values allow a function to send data back to the caller. A function can return any data type (int, str, list, dict, even another function).

Syntax & Key Points

```
def function(param1, param2):    # Parameters
    ...
    return result                # Return value
```

Important Points:

- Number of arguments must match number of parameters (unless using defaults)
- You can return multiple values using tuples
- Functions without return implicitly return None

Basic Example

```
def multiply(x, y):              # x and y are parameters
    return x * y

result = multiply(4, 5)          # 4 and 5 are arguments
print(result)                   # 20
```

Intermediate Example - Multiple Return

```
def get_user_info():
    name = "Keval"
    age = 25
    city = "Ahmedabad"
    return name, age, city      # Returns a tuple

info = get_user_info()
print(info)                    # ('Keval', 25, 'Ahmedabad')

# Unpacking
n, a, c = get_user_info()
print(n, a, c)                 # Keval 25 Ahmedabad
```

Default Arguments

Default arguments allow you to assign default values to parameters. If the caller doesn't provide a value, the default is used.

Default parameters must come **after** non-default parameters.

Syntax & Key Points

```
def function(param1, param2= default_value):
    ...
```

Rules:

- Default arguments are evaluated only once at function definition time
- Mutable defaults (like lists or dicts) can cause unexpected bugs

Basic Example

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")
```

```
greet("Keval")           # Hello, Keval!
greet("Rahul", "Good Morning") # Good Morning, Rahul!
```

Intermediate Example (Avoiding mutable default bug)

```
# Wrong way
def add_item(item, cart=[]):      # Dangerous!
    cart.append(item)
    return cart

# Correct way
def add_item(item, cart=None):
    if cart is None:
        cart = []
    cart.append(item)
    return cart
```

*args and kwargs

Topic Explanation

- args allows a function to accept **any number of positional arguments** as a tuple.
- *kwargs allows a function to accept **any number of keyword arguments** as a dictionary.

They provide flexibility when you don't know how many arguments will be passed.

Syntax & Key Points

```
def function(*args, **kwargs):
    ...
```

Rules:

- args must come before **kwargs
- You can mix normal parameters, *args, and **kwargs

Basic Example

```
def add_all(*numbers):
    return sum(numbers)

print(add_all(1, 2, 3, 4, 5))      # 15
```

Intermediate + Advanced Example

```
def student_info(name, *subjects, **details):
    print("Name:", name)
    print("Subjects:", subjects)      # tuple
    print("Details:", details)        # dict

student_info("Keval", "Math", "Science", "English",
            age=25, city="Ahmedabad", grade="A")

# Output:
# Name: Keval
```

```
# Subjects: ('Math', 'Science', 'English')
# Details: {'age': 25, 'city': 'Ahmedabad', 'grade': 'A'}
```

Lambda Functions

Lambda functions are small, anonymous (unnamed) functions defined in a single line. They are useful when you need a simple function for a short time (e.g., with `map()`, `filter()`).

Syntax & Key Points

```
lambda arguments: expression
```

Rules:

- Can take any number of arguments but only **one expression**
- Cannot contain statements like `if`, `for`, `return`
- Usually used with higher-order functions

Basic Example

```
square = lambda x: x ** 2
print(square(5))                # 25
```

Intermediate Example

```
add = lambda a, b: a + b
print(add(10, 20))              # 30

# With conditional
max_num = lambda x, y: x if x > y else y
print(max_num(15, 8))           # 15
```

Recursion

Recursion is when a function calls itself to solve smaller instances of the same problem. Every recursive function must have a base case to stop recursion.

Syntax & Key Points

```
def recursive_function(n):
    if n == 0:                # Base case
        return 1
    return n * recursive_function(n-1)  # Recursive call
```

Rules:

- Must have a base case to prevent infinite recursion
- Each recursive call uses stack memory (can cause stack overflow if too deep)

Basic Example - Factorial

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
```

```
print(factorial(5))          # 120
```

Intermediate Example - Fibonacci

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(6))          # 8
```

Higher-order Functions (map, filter, reduce)

Higher-order functions are functions that:

- Take other functions as arguments, or
- Return a function as result

map(), filter(), and reduce() are classic examples.

Syntax & Key Points

map() - Applies function to every item

```
map(function, iterable)
```

filter() - Filters items based on condition

```
filter(function, iterable)
```

reduce() - Reduces iterable to single value (needs functools)

```
from functools import reduce
reduce(function, iterable)
```

Example 1: map()

```
numbers = [1, 2, 3, 4, 5]

squared = list(map(lambda x: x**2, numbers))
print(squared)          # [1, 4, 9, 16, 25]
```

Example 2: filter()

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]

even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)      # [2, 4, 6, 8]
```

Example 3: reduce()

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
```

```
total = reduce(lambda x, y: x + y, numbers)
print(total)                                # 15
```

Modules & Packages

A **module** is a Python file (.py) containing functions, classes, and variables. **Importing** a module allows you to use code written in another file or from Python's standard library.

Modules promote code reusability and organization. Instead of writing everything in one file, you can split functionality into separate modules.

Why Modules are used:

- To organize large codebases
- To reuse existing code (standard library or third-party)
- To keep code clean and maintainable

Syntax & Key Points

```
import module_name
import module_name as alias
from module_name import specific_function
from module_name import *           # Not recommended
```

Important Rules:

- import statements are usually placed at the top of the file
- You can import the same module multiple times — Python loads it only once
- from ... import * pollutes the namespace and is generally avoided

Basic Example

```
import math

print(math.sqrt(16))      # 4.0
print(math.pi)            # 3.141592653589793
```

Intermediate Example

```
import math as m

print(m.pow(2, 3))        # 8.0
print(m.ceil(4.3))        # 5
```

Advanced Example

```
from math import sqrt, pow, pi

print(sqrt(25))           # 5.0
print(pow(3, 2))          # 9.0
print(pi)                 # 3.141592653589793
```

Creating Your Own Modules

You can create your own modules by writing Python code in a .py file and then importing it into other files. This is essential for building large applications.

A module can contain functions, classes, variables, and even executable code.

Syntax & Key Points

1. Create a file e.g., `utils.py`
2. Write functions/classes in it
3. Import it in another file using `import utils`

Rules:

- The filename becomes the module name (without `.py`)
- If the module is in the same folder, you can import directly
- Use `if __name__ == "__main__":` to run code only when the file is executed directly

Creating a module (`utils.py`)

```
# utils.py

def add(a, b):
    return a + b

def multiply(a, b):
    return a * b

def greet(name):
    return f"Hello, {name}!"

# This code runs only when file is executed directly
if __name__ == "__main__":
    print("This is utils module")
    print(add(10, 20))          # 30
```

Using the custom module (`main.py`)

```
# main.py
import utils

print(utils.add(5, 7))          # 12
print(utils.greet("Keval"))     # Hello, Keval!
```

Standard Libraries (Important Ones)

A. math Module

The math module provides mathematical functions and constants for advanced calculations.

Important Functions & Constants

```
import math

print(math.sqrt(36))            # 6.0
print(math.pow(2, 4))           # 16.0
print(math.ceil(4.2))           # 5
print(math.floor(4.8))          # 4
print(math.pi)                  # 3.141592653589793
print(math.e)                   # 2.718281828459045
print(math.factorial(5))        # 120
```

```
print(math.sin(math.radians(90))) # 1.0
print(math.gcd(48, 18))           # 6
```

B. random Module

The random module is used to generate random numbers, make random selections, and shuffle data.

Important Functions

```
import random

print(random.random())           # Random float between 0.0 and 1.0
print(random.randint(1, 100))    # Random integer between 1 and 100
print(random.choice(['Apple', 'Banana', 'Cherry'])) # Random item from list

numbers = [1, 2, 3, 4, 5]
random.shuffle(numbers)
print(numbers)                   # List is shuffled in-place

print(random.uniform(10.5, 20.5)) # Random float in range
```

C. datetime Module

The datetime module is used to work with dates, times, and time intervals.

Important Classes & Functions

```
from datetime import datetime, date, timedelta

# Current date and time
now = datetime.now()
print(now)                                # 2026-03-29 15:45:22.123456

print(now.strftime("%d-%m-%Y"))          # 29-03-2026
print(now.strftime("%A"))                 # Sunday

# Date arithmetic
future = now + timedelta(days=10)
print(future.strftime("%d %B %Y"))        # 08 April 2026

# Creating specific date
birthday = date(2000, 5, 15)
print(birthday)                           # 2000-05-15
```

D. os Module

The os module provides functions to interact with the operating system — file and directory operations, environment variables, etc.

Important Functions

```
import os

print(os.getcwd())                  # Current working directory

# Create directory
os.mkdir("test_folder")

# List files and folders
print(os.listdir())                 # List of files in current directory
```

```
# Check if path exists
print(os.path.exists("test_folder"))    # True

# Join paths safely (cross-platform)
full_path = os.path.join("folder", "file.txt")
print(full_path)
```

E. sys Module

The sys module provides access to Python interpreter variables and functions. It is commonly used for command-line arguments and system-specific parameters.

Important Functions & Variables

```
import sys

print(sys.version)          # Python version
print(sys.platform)        # 'win32', 'linux', etc.

# Command line arguments
print(sys.argv)            # List of arguments passed to script

# Exit program
# sys.exit("Error message")  # Uncomment to exit

print(sys.path)            # List of directories where Python looks for modules
```

Complete Summary of Usage Pattern

```
# Best practices for importing
import math
import random as rnd
from datetime import datetime, timedelta
import os
import sys

# Example combining multiple modules
def generate_report():
    now = datetime.now()
    random_id = random.randint(1000, 9999)

    print(f"Report Generated on: {now.strftime('%d %b %Y')}")
    print(f"Report ID: {random_id}")

generate_report()
```

Object-Oriented Programming (OOP)

1. Classes & Objects

Topic Explanation

A **class** is a blueprint or template for creating objects. It defines properties (variables) and behaviors (methods) that the objects will have.

An **object** is an instance of a class — a real entity created from the blueprint.

OOP helps in modeling real-world entities (like a Model, Dataset, NeuralNetwork) in a clean, reusable, and organized way.

Why Classes & Objects are used:

- To model complex real-world or ML concepts (e.g., a LinearRegression model).
- To achieve code reusability and better organization.
- Foundation for all other OOP concepts.

Syntax & Key Points

```
class ClassName:
    # class body
    pass
```

Rules:

- Class names use **PascalCase** (e.g., LinearRegression).
- Objects are created using ClassName().

Basic Example

```
class Car:
    pass

car1 = Car()          # Creating an object
car2 = Car()

print(type(car1))     # <class '__main__.Car'>
print(car1)           # <__main__.Car object at 0x...>
```

Intermediate Example – ML Context

```
class Dataset:
    def __init__(self, name):
        self.name = name

data1 = Dataset("MNIST")
data2 = Dataset("CIFAR10")

print(data1.name)     # MNIST
print(data2.name)     # CIFAR10
```

Advanced Example

```
class NeuralNetwork:
    def __init__(self, layers):
        self.layers = layers
        self.parameters = {}

nn = NeuralNetwork([784, 128, 64, 10])
print(f"Model has {len(nn.layers)} layers")    # Model has 4 layers
```

2. Constructors (__init__)

Topic Explanation

The __init__ method is a special **constructor** that is automatically called when a new object is created. It is used to initialize the object's attributes.\

Syntax & Key Points

```
def __init__(self, param1, param2):
    self.param1 = param1
    self.param2 = param2
```

Rules:

- self refers to the current object being created.
- __init__ is not a return type method (it returns None implicitly).

Basic Example

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s1 = Student("Keval", 25)
print(s1.name, s1.age)      # Keval 25
```

Advanced ML Example

```
class LinearRegression:
    def __init__(self, learning_rate=0.01, epochs=1000):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None

model = LinearRegression(learning_rate=0.001)
print(model.learning_rate)  # 0.001
```

3. Instance vs Class Variables

Topic Explanation

- **Instance Variables:** Unique to each object (defined inside __init__ using self).
- **Class Variables:** Shared by all objects of the class (defined directly inside the class).

Syntax & Key Points

```
class ClassName:
    class_variable = value          # Class variable

    def __init__(self):
        self.instance_variable = value  # Instance variable
```

Basic Example

```
class Employee:
    company = "xAI"                  # Class variable

    def __init__(self, name):
        self.name = name            # Instance variable

e1 = Employee("Keval")
```

```
e2 = Employee("Rahul")

print(e1.company)      # xAI
print(e2.company)      # xAI
Employee.company = "Tesla"
print(e1.company)      # Tesla (changed for all)
```

Advanced Example

```
class Model:
    total_models = 0          # Class variable

    def __init__(self, name):
        self.name = name
        Model.total_models += 1

m1 = Model("ResNet")
m2 = Model("BERT")
print(Model.total_models)    # 2
```

4. Methods (Instance, Class, Static)

Topic Explanation

- **Instance Method:** Works with instance (uses self).
- **Class Method:** Works with the class (uses cls). Defined with @classmethod.
- **Static Method:** Doesn't depend on instance or class (utility function). Defined with @staticmethod.

Instance Method

```
class Calculator:
    def __init__(self, value):
        self.value = value

    def add(self, x):          # Instance method
        return self.value + x

calc = Calculator(10)
print(calc.add(5))           # 15
```

Class Method

```
class Model:
    version = "1.0"

    @classmethod
    def get_version(cls):
        return f"Model version: {cls.version}"

print(Model.get_version())    # Model version: 1.0
```

Static Method

```
class MathUtils:
    @staticmethod
    def is_even(n):
        return n % 2 == 0
```

```
print(MathUtils.is_even(10))          # True
```

5. Inheritance

Topic Explanation

Inheritance allows a class (**child**) to inherit attributes and methods from another class (**parent**). It promotes code reuse.

Types: Single, Multiple, Multilevel, Hierarchical, Hybrid.

Syntax & Key Points

```
class Child(Parent):  
    ...
```

Single Inheritance

```
class Animal:  
    def speak(self):  
        print("Animal speaks")  
  
class Dog(Animal):  
    def bark(self):  
        print("Woof!")  
  
d = Dog()  
d.speak()      # Animal speaks  
d.bark()       # Woof!
```

Multiple Inheritance

```
class Flyable:  
    def fly(self):  
        print("I can fly!")  
  
class Swimmable:  
    def swim(self):  
        print("I can swim!")  
  
class Duck(Flyable, Swimmable):  
    pass  
  
d = Duck()  
d.fly()        # I can fly!  
d.swim()       # I can swim!
```

Multilevel Inheritance

```
class Vehicle:  
    def start(self):  
        print("Vehicle started")  
  
class Car(Vehicle):
```

```

    def drive(self):
        print("Car is driving")

class SportsCar(Car):
    def turbo(self):
        print("Turbo mode activated!")

sc = SportsCar()
sc.start()
sc.drive()
sc.turbo()

```

6. Polymorphism

Topic Explanation

Polymorphism means "many forms". The same method name can behave differently depending on the object.

Achieved via:

- Method Overriding
- Operator Overloading (via dunder methods)

Method Overriding

```

class Shape:
    def area(self):
        return 0

class Rectangle(Shape):
    def __init__(self, l, b):
        self.length = l
        self.breadth = b

    def area(self):
        # Overriding
        return self.length * self.breadth

r = Rectangle(5, 6)
print(r.area())
# 30

```

Operator Overloading (Polymorphism)

```

class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        # Dunder method for +
        return Vector(self.x + other.x, self.y + other.y)

v1 = Vector(2, 3)
v2 = Vector(5, 7)
v3 = v1 + v2
print(v3.x, v3.y)
# 7 10

```

7. Encapsulation

Topic Explanation

Encapsulation is wrapping data and methods into a single unit (class) and restricting direct access to some components.

In Python, we use:

- `_` → Protected (convention)
- `__` → Private (name mangling)

Examples

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance        # Private

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance          # Getter

acc = BankAccount(1000)
acc.deposit(500)
print(acc.get_balance())               # 1500

# Direct access not possible
# print(acc.__balance)                 # AttributeError
```

8. Abstraction

Topic Explanation

Abstraction hides complex implementation details and shows only essential features.

Achieved using **Abstract Base Classes (ABC)** from `abc` module.

Syntax & Key Points

```
from abc import ABC, abstractmethod
```

Example

```
from abc import ABC, abstractmethod

class Model(ABC):
    @abstractmethod
    def train(self):
        pass

    @abstractmethod
    def predict(self):
        pass

class LinearRegression(Model):
    def train(self):
        print("Training Linear Regression...")
```

```
def predict(self):
    print("Making prediction...")

model = LinearRegression()
model.train()
model.predict()
```

9. Magic / Dunder Methods

Topic Explanation

Dunder (Double Underscore) methods are special methods that start and end with `__`. They allow you to define how objects behave with built-in functions and operators.

Important Dunder Methods

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        # Used by print()
        return f"Book: {self.title} ({self.pages} pages)"

    def __repr__(self):
        # Used by repr()
        return f"Book('{self.title}', {self.pages})"

    def __len__(self):
        # Used by len()
        return self.pages

    def __add__(self, other):
        # Used by +
        return Book(f"{self.title} + {other.title}", self.pages + other.pages)

b1 = Book("Python OOP", 300)
b2 = Book("ML Basics", 450)

print(b1)          # Book: Python OOP (300 pages)
print(len(b1))     # 300
print(b1 + b2)     # Book: Python OOP + ML Basics (750 pages)
print(repr(b1))    # Book('Python OOP', 300)
```

File Handling

File Handling is the ability to read from and write to files on your computer. Python provides built-in functions to work with different file types like text (.txt), CSV (.csv), and JSON (.json).

File handling is essential because:

- Most real-world data comes from files (datasets, logs, configurations).
- You often need to save results, models, or reports.
- In Machine Learning, you frequently read large datasets and save trained models.

Key Concepts:

- **File modes:** Control how you open the file (read, write, append, etc.).
- **Context Manager** (with statement): Safest and recommended way to handle files.
- Handling different formats: plain text, CSV, and JSON.

Syntax & Key Points

Mode	Description	Use Case
'r'	Read (default)	Reading existing files
'w'	Write (overwrites if file exists)	Creating new files
'a'	Append (adds to end)	Adding logs
'r+'	Read and Write	Modifying existing files
'x'	Create (fails if file exists)	Safe new file creation

Basic Syntax

```
with open('filename.txt', mode) as file:
    # operations
```

Important Rules:

- Always use with open() — it automatically closes the file.
- File paths can be relative or absolute.
- Encoding 'utf-8' is recommended for text files.

A. Reading Files

Example 1: Reading Text File (.txt)

```
# Reading entire file
with open('sample.txt', 'r', encoding='utf-8') as file:
    content = file.read()
    print(content)

# Reading line by line
with open('sample.txt', 'r', encoding='utf-8') as file:
    for line in file:
        print(line.strip())           # strip() removes newline
```

Example 2: Reading CSV File

```
import csv

with open('data.csv', 'r', encoding='utf-8') as file:
    reader = csv.reader(file)
    next(reader)           # Skip header row
    for row in reader:
        print(row)         # Each row is a list
```

Example 3: Reading JSON File

```
import json

with open('config.json', 'r', encoding='utf-8') as file:
    data = json.load(file)    # Converts JSON to Python dict

print(data['name'])
print(data['age'])
```

B. Writing Files

Example 4: Writing to Text File

```
# Write mode (overwrites)
with open('output.txt', 'w', encoding='utf-8') as file:
    file.write("Hello Python!\n")
    file.write("This is line 2.\n")

# Append mode
with open('output.txt', 'a', encoding='utf-8') as file:
    file.write("This line is appended.\n")
```

Example 5: Writing CSV File

```
import csv

data = [
    ['Name', 'Age', 'City'],
    ['Keval', 25, 'Ahmedabad'],
    ['Priya', 23, 'Mumbai']
]

with open('students.csv', 'w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    writer.writerows(data)
```

Example 6: Writing JSON File

```
import json

student = {
    "name": "Keval",
    "age": 25,
    "skills": ["Python", "ML", "Data Science"],
    "active": True
}

with open('student.json', 'w', encoding='utf-8') as file:
    json.dump(student, file, indent=4)    # indent makes it readable
```

Context Manager (with open())

Why use with statement?

- Automatically closes the file after use (even if error occurs).
- Cleaner and safer than manual file.close().

Without Context Manager (Not Recommended)

```
file = open('data.txt', 'r')
content = file.read()
file.close()          # Easy to forget this
```

With Context Manager (Best Practice)

```
with open('data.txt', 'r', encoding='utf-8') as file:
    content = file.read()
# File is automatically closed here
```

Working with Large Files

When dealing with very large files (ML datasets, logs), you should **never** load the entire file into memory.

Best Practices for Large Files

Example 7: Reading Large Text File Line by Line

```
with open('large_dataset.csv', 'r', encoding='utf-8') as file:
    for line in file:
        # Process one line at a time
        row = line.strip().split(',')
        # Do your processing here
        if int(row[2]) > 100:          # Example condition
            print(row)
```

Example 8: Using CSV DictReader for Large Files

```
import csv

with open('large_data.csv', 'r', encoding='utf-8') as file:
    reader = csv.DictReader(file)
    for row in reader:
        # row is a dictionary
        print(row['name'], row['score'])
```

Memory Efficient Tip:

- Use `for line in file:` instead of `file.read()`
- Process data in chunks
- Consider libraries like pandas for very large structured data (we'll cover later)

All Important File Operations

```
# File Information
with open('file.txt', 'r') as f:
    print(f.name)          # File name
    print(f.mode)          # Mode used
    print(f.closed)        # False

# Check if file exists
import os
print(os.path.exists('file.txt'))

# Get file size
print(os.path.getsize('file.txt'), "bytes")
```

Edge Cases:

- File not found → `FileNotFoundError`
- Permission issues → `PermissionError`
- Encoding errors with special characters → Use `encoding='utf-8'`
- Writing to same file while reading → Use different modes carefully

Exception Handling

Exception Handling is a mechanism in Python to handle runtime errors gracefully so that the program doesn't crash unexpectedly.

An **exception** is an error that occurs during program execution (e.g., dividing by zero, file not found, invalid input). Without handling, the program stops abruptly.

Python provides a clean way to **catch**, **handle**, and **recover** from these errors using try, except, else, and finally blocks.

Why Exception Handling is important:

- Makes programs robust and user-friendly.
- Prevents sudden crashes in production code.
- Allows graceful error reporting and recovery.
- Essential in real-world applications like ML pipelines, web apps, and data processing.

Real-world use cases:

- Handling invalid user input
- Dealing with missing files or network failures
- Gracefully managing API errors
- Preventing training crashes in Machine Learning scripts

Syntax & Key Points

Basic Structure

```
try:
    # Code that might raise an exception
    risky_code()

except ExceptionType as e:
    # Code to handle the exception
    print("Error occurred:", e)

else:
    # Executes only if no exception occurred
    print("Success!")

finally:
    # Always executes, whether exception occurred or not
    print("Cleanup code")
```

Important Rules:

- try block is mandatory.
- You can have multiple except blocks.
- else runs only if no exception was raised.
- finally always runs (used for cleanup like closing files).
- Use specific exceptions first, then general ones (Exception).

A. try / except / else / finally

Basic Example

```
try:
    num = int(input("Enter a number: "))    # User enters "abc"
    result = 100 / num
    print("Result:", result)
```

```

except ValueError:
    print("Error: Please enter a valid number!")

except ZeroDivisionError:
    print("Error: Cannot divide by zero!")

else:
    print("Operation completed successfully!")

finally:
    print("This will always run - Cleanup done.")

```

Intermediate Example - File Handling with Exception

```

try:
    with open('non_existent_file.txt', 'r') as file:
        content = file.read()
        print(content)

except FileNotFoundError as e:
    print(f"File not found: {e}")

except Exception as e:          # Catch any other error
    print(f"Unexpected error: {e}")

else:
    print("File read successfully!")

finally:
    print("File operation attempt completed.")

```

Advanced Example - ML Context

```

def train_model(data_path):
    try:
        with open(data_path, 'r') as f:
            data = f.read()
            # Simulate training
            print(f"Training started with {len(data)} characters...")

    except FileNotFoundError:
        print(f"Dataset not found at {data_path}. Please check the path.")

    except PermissionError:
        print("Permission denied. Cannot read the dataset.")

    except Exception as e:
        print(f"Training failed due to: {type(e).__name__} - {e}")

    finally:
        print("Training pipeline finished.")

train_model("missing_dataset.csv")

```

Custom Exceptions

Sometimes built-in exceptions are not descriptive enough. You can create your own **custom exceptions** by inheriting from the Exception class.

This makes error handling more meaningful and specific to your application.

Syntax & Key Points

```
class CustomErrorName(Exception):  
    pass
```

Basic Custom Exception

```
class NegativeNumberError(Exception):  
    def __init__(self, message="Number cannot be negative"):  
        self.message = message  
        super().__init__(self.message)  
  
def check_positive(num):  
    if num < 0:  
        raise NegativeNumberError(f"Received negative number: {num}")  
    return num * 2  
  
try:  
    result = check_positive(-5)  
except NegativeNumberError as e:  
    print(e)                                # Received negative number: -5
```

Advanced Custom Exception (ML Context)

```
class ModelTrainingError(Exception):  
    """Custom exception for ML model training failures"""  
    def __init__(self, model_name, reason):  
        self.model_name = model_name  
        self.reason = reason  
        super().__init__(f"Training failed for {model_name}: {reason}")  
  
def train_neural_net(model_name, data_size):  
    if data_size < 1000:  
        raise ModelTrainingError(model_name, "Insufficient training data")  
    print(f"Training {model_name} successful!")  
  
try:  
    train_neural_net("ResNet50", 500)  
except ModelTrainingError as e:  
    print(e)
```

Debugging Techniques

Debugging is the process of finding and fixing bugs in your code. Python provides several built-in tools and techniques.

Important Debugging Techniques

1. Using print() statements

```
def divide(a, b):  
    print(f"Debug: a = {a}, b = {b}")    # Temporary debug  
    return a / b
```

2. Using assert for debugging

```
def calculate_average(numbers):  
    assert len(numbers) > 0, "List cannot be empty!"  
    return sum(numbers) / len(numbers)
```

3. Using pdb (Python Debugger)

```
import pdb

def debug_function():
    x = 10
    y = 0
    pdb.set_trace()      # Set breakpoint
    result = x / y
    return result
```

4. Exception Information

```
try:
    10 / 0
except Exception as e:
    print(type(e).__name__)    # ZeroDivisionError
    print(e)                  # division by zero
    import traceback
    traceback.print_exc()     # Full stack trace
```

5. Common Debugging Practices

- Use meaningful variable names
- Add comments explaining complex logic
- Write small functions and test them individually
- Use logging instead of print() in production

6. All Important Built-in Exceptions

Here are the most commonly used exceptions:

- ValueError → Wrong value type
- TypeError → Wrong data type
- ZeroDivisionError → Division by zero
- FileNotFoundError → File doesn't exist
- IndexError → List index out of range
- KeyError → Dictionary key not found
- AttributeError → Object has no such attribute
- ImportError / ModuleNotFoundError → Module not found

Iterators & Generators

Iterators and **Generators** are powerful concepts in Python used for **lazy evaluation** — meaning they generate values one at a time instead of storing everything in memory at once.

- An **Iterator** is an object that can be iterated (looped) over. It implements the iterator protocol (`__iter__()` and `__next__()`).
- A **Generator** is a special type of iterator that is written using functions with the `yield` keyword. It allows you to create iterators in a very simple and memory-efficient way.

Why Iterators & Generators are important:

- They are **memory efficient** — especially useful when working with large datasets (common in Machine Learning and Data Science).
- They allow **lazy loading** — values are produced only when needed.
- They make code cleaner and more Pythonic.

Real-world use cases:

- Reading very large files line by line
- Generating infinite sequences
- Processing large ML datasets without loading everything into RAM
- Creating custom data pipelines

Iterators

An iterator is any object that implements:

- `__iter__()` → Returns the iterator object itself
- `__next__()` → Returns the next value and raises `StopIteration` when done

Lists, tuples, strings, files, etc. are all **iterable** (they can give you an iterator via `iter()`).

Syntax & Key Points

```
iterator = iter(iterable)    # Get iterator from iterable
next(iterator)               # Get next value
```

Basic Example - Manual Iterator

```
numbers = [10, 20, 30, 40]

it = iter(numbers)          # Get iterator

print(next(it))              # 10
print(next(it))              # 20
print(next(it))              # 30
print(next(it))              # 40
# print(next(it))           # Raises StopIteration
```

Custom Iterator Class

```
class EvenNumbers:
    def __init__(self, max_num):
        self.max_num = max_num
        self.current = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= self.max_num:
            raise StopIteration
        self.current += 2
        return self.current - 2

even_iter = EvenNumbers(10)

for num in even_iter:
    print(num)                # 0 2 4 6 8
```

yield Keyword & Generators

Topic Explanation

The `yield` keyword turns a normal function into a **generator function**.

When a generator function is called, it **does not execute** the body immediately. Instead, it returns a **generator object**. Each time `next()` is called on it, the function runs until it hits `yield`, returns that value, and **pauses** its state.

This makes generators extremely memory efficient.

Syntax & Key Points

```
def generator_function():
    yield value1
    yield value2
```

Key Differences from return:

- `return` ends the function completely.
- `yield` pauses the function and saves its state.

Basic Generator Example

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count          # Pause and return value
        count += 1

gen = count_up_to(5)

print(next(gen))            # 1
print(next(gen))            # 2
print(next(gen))            # 3

for num in gen:              # Remaining values
    print(num)               # 4 5
```

Intermediate Example - Infinite Generator

```
def infinite_counter():
    count = 1
    while True:
        yield count
        count += 1

counter = infinite_counter()

print(next(counter))        # 1
print(next(counter))        # 2
print(next(counter))        # 3
```

Advanced ML-style Generator

```
def data_generator(batch_size=32):
    data = list(range(1000))    # Imagine this is a huge dataset
    i = 0
    while True:
        batch = data[i:i + batch_size]
        if not batch:
            break
        yield batch              # Yield one batch at a time
        i += batch_size
```

```
for batch in data_generator(100):
    print(f"Processing batch of size: {len(batch)}")
```

Generator Expressions

Topic Explanation

Generator expressions are like **list comprehensions**, but they use `()` instead of `[]`. They create generators (lazy) instead of lists (eager).

They are more memory efficient than list comprehensions when you don't need all values at once.

Syntax & Key Points

```
gen_expr = (expression for item in iterable if condition)
```

Basic Generator Expression

```
# List comprehension (stores everything in memory)
squares_list = [x**2 for x in range(1000000)]

# Generator expression (very memory efficient)
squares_gen = (x**2 for x in range(1000000))

print(type(squares_gen))          # <class 'generator'>

print(next(squares_gen))          # 0
print(next(squares_gen))          # 1
```

Intermediate Example

```
# Generator expression with condition
even_squares = (x**2 for x in range(20) if x % 2 == 0)

for val in even_squares:
    print(val)                    # 0 4 16 36 64 ...
```

Advanced Example - Memory Comparison

```
import sys

# List vs Generator
big_list = [x * 2 for x in range(1000000)]
big_gen = (x * 2 for x in range(1000000))

print("List memory:", sys.getsizeof(big_list))      # Much larger
print("Generator memory:", sys.getsizeof(big_gen))  # Very small
```

All Important Points & Differences

Feature	Iterator	Generator	Generator Expression
Created using	<code>__iter__</code> and <code>__next__</code>	<code>yield</code> keyword	<code>()</code> syntax
Memory Usage	Low	Very Low	Very Low
Code Simplicity	Complex	Simple	Very Simple
State Saving	Manual	Automatic	Automatic
Best For	Custom iteration	Lazy sequences, pipelines	Quick lazy operations

Key Advantages of Generators:

- Memory efficient (produce one value at a time)
- Can represent infinite sequences
- Faster startup time

Decorators

Decorators are one of the most powerful and Pythonic features in Python.

A **decorator** is a function that takes another function as input, adds some functionality to it, and returns a new function **without modifying the original function's code**.

Think of a decorator as a **wrapper** — it wraps around a function and adds extra behavior (like logging, timing, authentication, validation, etc.) before or after the original function runs.

Decorators use the `@decorator_name` syntax above a function definition.

Why Decorators are used:

- To add common functionality (logging, timing, caching, authorization) without repeating code.
- To keep functions clean and focused on their main logic.
- To separate cross-cutting concerns (logging, error handling, etc.).
- Very important in frameworks like Flask, Django, and in Machine Learning pipelines.

Real-world use cases:

- Logging function calls and execution time
- Checking user authentication before running a function
- Caching expensive computations
- Input validation
- Rate limiting API calls
- Timing model training in ML

Basic Syntax

```
def decorator_name(func):
    def wrapper(*args, **kwargs):
        # Code before function call
        result = func(*args, **kwargs)
        # Code after function call
        return result
    return wrapper
```

Using Decorator:

```
@decorator_name
def my_function():
    ...
```

A. Basic Function Decorator

Example 1: Simple Decorator

```
def simple_decorator(func):
    def wrapper():
        print("Before function execution")
        func()
        print("After function execution")
    return wrapper
```

```
@simple_decorator
def say_hello():
    print("Hello, World!")

say_hello()
```

Output:

```
Before function execution
Hello, World!
After function execution
```

B. Decorator with Arguments

Example 2: Decorator that accepts arguments

```
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Hello, {name}!")

greet("Keval")
```

Output:

```
Hello, Keval!
Hello, Keval!
Hello, Keval!
```

C. Real-World Useful Decorators

Example 3: Timer Decorator (Very Important)

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"Function '{func.__name__}' took {end - start:.4f} seconds to run")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1.5)
    print("Function completed")
```

```
slow_function()
```

Example 4: Logging Decorator

```
def log_calls(func):
    def wrapper(*args, **kwargs):
        print(f"Calling function: {func.__name__}")
        print(f"Arguments: {args}, {kwargs}")
        result = func(*args, **kwargs)
        print(f"Function {func.__name__} finished")
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b

result = add(10, 20)
print("Result:", result)
```

Example 5: Authentication Decorator (Real-world use)

```
def require_login(func):
    def wrapper(user, *args, **kwargs):
        if not user.get("logged_in"):
            print("Error: User must be logged in to access this function!")
            return None
        return func(user, *args, **kwargs)
    return wrapper

@require_login
def view_dashboard(user):
    print(f"Welcome to dashboard, {user['name']}!")

# Test
user1 = {"name": "Keval", "logged_in": True}
user2 = {"name": "Rahul", "logged_in": False}

view_dashboard(user1)    # Works
view_dashboard(user2)    # Shows error
```

Example 6: ML Pipeline Decorator (Advanced Use Case)

```
def track_training(func):
    def wrapper(*args, **kwargs):
        print("🚀 Starting model training...")
        start = time.time()

        result = func(*args, **kwargs)

        duration = time.time() - start
        print(f"✅ Training completed in {duration:.2f} seconds")
        print(f"Final Accuracy: {result:.4f}")
        return result
    return wrapper

@track_training
def train_model(epochs=10):
    # Simulate training
```

```
time.sleep(2)
return 0.9456

accuracy = train_model(epochs=50)
```

Preserving Function Metadata (@wraps)

Important Best Practice

```
from functools import wraps

def timer(func):
    @wraps(func)                                # Very Important!
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end-start:.4f} seconds")
        return result
    return wrapper
```

Without @wraps, help() and function name/docstring get lost.

Multiple Decorators (Stacking)

You can apply multiple decorators. They are applied from bottom to top.

```
@timer
@log_calls
def process_data():
    print("Processing data...")
    return "Done"

process_data()
```

Real Use Cases Summary

Decorator Use Case	Description
@timer	Measure execution time
@log_calls	Log function calls and arguments
@require_login	Check authentication
@cache (from functools)	Memoization / caching results
@lru_cache	LRU Cache for expensive functions
Pipeline decorators	Track ML training, validation, preprocessing
Error handling decorators	Centralized exception logging

Context Managers

Context Managers are Python objects that allow you to manage resources (like files, database connections, locks, network sessions, etc.) efficiently and safely.

The most common use of a context manager is the with statement:

```
with open('file.txt') as f:
    data = f.read()
```

The biggest advantage is that the resource is **automatically cleaned up** (e.g., file is closed) even if an exception occurs.

Context managers ensure:

- Setup code runs before entering the block
- Cleanup code runs after exiting the block (guaranteed)
- Exception safety

Why Context Managers are important:

- Prevent resource leaks (forgotten file closes, unclosed database connections)
- Make code cleaner and more readable
- Very useful in Machine Learning (managing model training sessions, GPU memory, data loaders, etc.)

How Context Managers Work (The Protocol)

A context manager must implement two special methods (the **context management protocol**):

- `__enter__()` → Called when entering the with block. Usually returns the resource.
- `__exit__()` → Called when exiting the with block. Handles cleanup and exceptions.

Built-in Context Managers (Quick Recap)

```
# File handling (most common)
with open('data.txt', 'r', encoding='utf-8') as file:
    content = file.read()

# Multiple context managers
with open('input.txt') as infile, open('output.txt', 'w') as outfile:
    outfile.write(infile.read())
```

Creating Custom Context Managers

There are two main ways to create custom context managers:

Method 1: Using a Class (Most Powerful)

Example 1: Simple Custom Context Manager

```
class Timer:
    def __enter__(self):
        self.start = time.time()
        print("Timer started...")
        return self                # Value returned to 'as' variable

    def __exit__(self, exc_type, exc_value, traceback):
        self.end = time.time()
        duration = self.end - self.start
        print(f"Timer stopped. Duration: {duration:.4f} seconds")

        # Return False means exceptions are NOT suppressed
        return False

# Using the custom context manager
with Timer():
    time.sleep(1.2)
    print("Some heavy computation...")
```

Example 2: Database Connection Simulator (Realistic Use)

```

class DatabaseConnection:
    def __init__(self, db_name):
        self.db_name = db_name

    def __enter__(self):
        print(f"Connecting to database: {self.db_name}")
        self.connection = f"ActiveConnection-{self.db_name}"
        return self.connection          # Returned to 'as' variable

    def __exit__(self, exc_type, exc_value, traceback):
        print(f"Closing connection to {self.db_name}")
        self.connection = None

        # If exception occurred
        if exc_type is not None:
            print(f"Error occurred: {exc_type.__name__}")
        return False                    # Do not suppress exception

# Usage
with DatabaseConnection("ml_training_db") as conn:
    print(f"Using connection: {conn}")
    # Simulate some database operation
    # raise ValueError("Test error")    # Try uncommenting this

```

Example 3: ML Training Session Context Manager (Advanced)

```

class TrainingSession:
    def __enter__(self):
        print("🚀 Starting ML Training Session...")
        print("GPU memory allocated")
        print("Model weights initialized")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("🧹 Cleaning up training session...")
        print("Releasing GPU memory")
        print("Saving logs...")

        if exc_type is not None:
            print(f"❌ Training failed with {exc_type.__name__}")
        else:
            print(f"✅ Training completed successfully")

        return False

# Usage in ML context
with TrainingSession():
    print("Training model for 50 epochs...")
    # Simulate long training
    time.sleep(1.5)
    # raise RuntimeError("Out of memory")    # Try this too

```

Method 2: Using @contextmanager Decorator (Simpler)

This is often preferred for simple use cases.

```

from contextlib import contextmanager
import time

```

```

@contextmanager
def timer_context():
    start = time.time()
    print("Timer started...")
    try:
        yield "Timer Active"          # Value passed to 'as' variable
    finally:
        end = time.time()
        print(f"Timer finished in {end - start:.4f} seconds")

# Usage
with timer_context() as t:
    print("Inside context:", t)
    time.sleep(0.8)

```

Example 4: Temporary Directory Context Manager

```

from contextlib import contextmanager
import tempfile
import os

@contextmanager
def temp_directory():
    dir_path = tempfile.mkdtemp()
    try:
        yield dir_path
    finally:
        print(f"Cleaning up temporary directory: {dir_path}")
        # os.rmdir(dir_path)  # Uncomment to actually delete

with temp_directory() as tmp_dir:
    print(f"Working in temporary directory: {tmp_dir}")
    with open(f"{tmp_dir}/test.txt", "w") as f:
        f.write("Temporary data")

```

Important Points & Best Practices

- `__exit__()` receives three arguments: `exc_type`, `exc_value`, `traceback`
- If `__exit__()` returns `True`, it **suppresses** the exception
- Always return `False` unless you intentionally want to suppress exceptions
- Use `@contextmanager` for simple cases
- Use Class-based context managers when you need more control or state

Real Use Cases in Practice

1. **Resource Management** — Files, Database connections, Network sockets
2. **Timing & Profiling** — Measure training time in ML
3. **Temporary Resources** — Temporary folders, mock environments
4. **Transaction Management** — Database transactions (commit/rollback)
5. **Lock Management** — Threading/multiprocessing locks
6. **ML Experiment Tracking** — Log metrics, save checkpoints automatically

Multithreading & Multiprocessing

Multithreading and **Multiprocessing** are two ways to achieve **concurrency** (doing multiple things at the same time) in Python.

- **Multithreading:** Multiple threads within a single process. Threads share the same memory space.
- **Multiprocessing:** Multiple independent processes. Each process has its own memory space and Python interpreter.

The GIL (Global Interpreter Lock) is the most important concept that differentiates them in CPython (the standard Python implementation).

Why these concepts matter:

- Python is widely used in **I/O-bound** tasks (web scraping, API calls, file handling) and **CPU-bound** tasks (data processing, model training, image processing).
- Proper use of threading/multiprocessing can dramatically improve performance.
- Misuse can lead to slower code, bugs, or high memory usage.

Real-world use cases:

- Downloading multiple files simultaneously (threading)
- Parallel data preprocessing for ML (multiprocessing)
- Training multiple models in parallel
- Web servers handling multiple requests

The GIL Concept (Global Interpreter Lock)

Explanation

The **GIL** is a mutex (lock) that protects access to Python objects. It ensures that **only one thread can execute Python bytecode at a time** in a single process.

- It was added to simplify memory management (reference counting).
- Even in 2026, the GIL is still present in the standard CPython interpreter (though efforts to remove it or make it optional continue).
- **Impact:** Multithreading **does not** give true parallelism for CPU-bound tasks in Python.

Key Takeaways:

- **I/O-bound tasks** (waiting for network, disk, database) → Threading is useful because threads release the GIL while waiting.
- **CPU-bound tasks** (heavy calculations, loops, ML training loops) → Threading gives little to no speedup due to GIL. Use **Multiprocessing** instead.

Multithreading (threading module)

Explanation

The threading module allows you to create and manage threads.

Syntax & Key Points

```
import threading

thread = threading.Thread(target=function, args=(arg1, arg2))
thread.start()
thread.join()          # Wait for thread to finish
```

Basic Example

```
import threading
import time

def worker(name):
    print(f"Worker {name} started")
```

```

        time.sleep(2)
        print(f"Worker {name} finished")

t1 = threading.Thread(target=worker, args=("A",))
t2 = threading.Thread(target=worker, args=("B",))

t1.start()
t2.start()

t1.join()
t2.join()

print("All threads completed")

```

Example with ThreadPoolExecutor (Recommended)

```

from concurrent.futures import ThreadPoolExecutor
import time

def download_file(file_id):
    print(f"Downloading file {file_id}...")
    time.sleep(1.5)
    return f"file_{file_id}.txt"

with ThreadPoolExecutor(max_workers=5) as executor:
    results = executor.map(download_file, range(10))

for result in results:
    print(result)

```

Multiprocessing (multiprocessing module)

Explanation

The multiprocessing module creates separate processes, each with its own Python interpreter and memory space. This **bypasses the GIL** and allows true parallelism on multi-core CPUs.

Syntax & Key Points

```

from multiprocessing import Process, Pool

process = Process(target=function, args=(arg,))
process.start()
process.join()

```

Basic Example

```

from multiprocessing import Process
import time

def cpu_heavy_task(name):
    print(f"Process {name} started")
    # Simulate heavy CPU work
    total = 0
    for i in range(5_000_000):
        total += i
    print(f"Process {name} finished. Sum = {total}")

```

```
if __name__ == "__main__":          # Very important on Windows
    p1 = Process(target=cpu_heavy_task, args=("P1",))
    p2 = Process(target=cpu_heavy_task, args=("P2",))

    p1.start()
    p2.start()

    p1.join()
    p2.join()

    print("All processes completed")
```

Recommended: ProcessPoolExecutor

```
from concurrent.futures import ProcessPoolExecutor

def square(n):
    return n * n

if __name__ == "__main__":
    numbers = range(100000)

    with ProcessPoolExecutor(max_workers=4) as executor:
        results = list(executor.map(square, numbers))

    print("First 10 squares:", results[:10])
```

Comparison: Threading vs Multiprocessing

Aspect	Threading	Multiprocessing
Memory	Shared memory	Separate memory (higher usage)
GIL Impact	Affected by GIL	Bypasses GIL
Best For	I/O-bound tasks	CPU-bound tasks
Speed (CPU)	Limited speedup	True parallelism
Communication	Easy (shared variables + locks)	Harder (requires queues, pipes)
Overhead	Low	High (process creation cost)
Debugging	Easier	Harder
Recommended Tool	ThreadPoolExecutor	ProcessPoolExecutor

Rule of Thumb:

- Mostly **waiting** (I/O) → Use **Threading**
- Mostly **computing** (CPU) → Use **Multiprocessing**

Important Notes & Best Practices

- Always protect the entry point with if __name__ == "__main__": when using multiprocessing (especially on Windows).
- Use concurrent.futures module (ThreadPoolExecutor / ProcessPoolExecutor) instead of raw Thread/Process for most cases.
- For shared data in multiprocessing, use Queue, Pipe, or Manager.
- Be careful with shared mutable state in threading — use Lock, RLock, Semaphore, etc.
- In Machine Learning: Use multiprocessing for data loading/preprocessing and threading for I/O operations.

Async Programming (Asynchronous Programming)

1. Topic Explanation

Asynchronous Programming allows your program to handle multiple tasks **concurrently** without using multiple threads or processes.

Instead of waiting (blocking) for an operation to complete (like network requests, file I/O, database queries, or API calls), the program can move on to other tasks and come back when the operation is ready.

Python's modern way to do this is using **async / await** syntax along with the **asyncio** library.

Key Difference from Previous Concepts:

- **Threading**: Uses multiple threads (affected by GIL for CPU work)
- **Multiprocessing**: Uses multiple processes (high memory usage)
- **Asyncio**: Uses **single thread + cooperative multitasking** — very efficient for I/O-bound tasks

Why Async Programming is Important:

- Extremely efficient for **I/O-bound** tasks (network calls, web scraping, APIs, file operations).
- Handles thousands of concurrent connections with low memory and CPU usage.
- Widely used in web frameworks (FastAPI, Starlette), web scraping, chatbots, and real-time systems.
- Becoming increasingly important in modern Python development.

When to use Async:

- Good → Making many HTTP requests, reading multiple files, database queries.
- Not Good → Heavy CPU computations (use multiprocessing instead).

2. async / await Basics

Syntax & Key Points

- **async def** → Defines an **asynchronous function** (coroutine)
- **await** → Used to wait for a coroutine or awaitable object without blocking
- You **cannot** call an async function directly — you must await it or run it through an event loop.

Example 1: Basic async / await

```
import asyncio

async def fetch_data():
    print("Start fetching data...")
    await asyncio.sleep(2)      # Simulate network delay (non-blocking)
    print("Data fetched successfully!")
    return "Important Data"

async def main():
    print("Main started")
    data = await fetch_data()   # Wait for fetch_data to complete
    print("Received:", data)
    print("Main finished")

# Run the async program
asyncio.run(main())
```

Output:

```
Main started
Start fetching data...
Data fetched successfully!
```

```
Received: Important Data
Main finished
```

3. asyncio Basics

Core Concepts

1. **Coroutine:** A function defined with `async def`
2. **Event Loop:** The core of `asyncio` — manages and schedules coroutines
3. **Task:** A wrapper around a coroutine that runs concurrently
4. **Awaitable:** Anything you can await (coroutine, Task, Future)

Important asyncio Functions

Example 2: Running Multiple Tasks Concurrently

```
import asyncio

async def task1():
    await asyncio.sleep(2)
    print("Task 1 completed")

async def task2():
    await asyncio.sleep(1)
    print("Task 2 completed")

async def task3():
    await asyncio.sleep(3)
    print("Task 3 completed")

async def main():
    # Create tasks to run concurrently
    t1 = asyncio.create_task(task1())
    t2 = asyncio.create_task(task2())
    t3 = asyncio.create_task(task3())

    print("All tasks started")
    await t1
    await t2
    await t3
    print("All tasks finished")

asyncio.run(main())
```

Example 3: Using `asyncio.gather()` (Most Common Pattern)

```
import asyncio

async def download_file(file_id):
    print(f"Downloading file {file_id}...")
    await asyncio.sleep(1.5)          # Simulate download time
    return f"file_{file_id}.jpg"

async def main():
    files = range(1, 11)              # 10 files

    # Run all downloads concurrently
    results = await asyncio.gather(
        *[download_file(i) for i in files]
```

```
)

print("All downloads completed!")
for result in results:
    print(result)

asyncio.run(main())
```

Example 4: Simulated API Calls (Very Common Pattern)

```
import asyncio
import random

async def call_api(endpoint):
    print(f"Calling API: {endpoint}")
    await asyncio.sleep(random.uniform(0.5, 2.0))    # Simulate network latency
    return f"Response from {endpoint}"

async def main():
    endpoints = ["users", "products", "orders", "analytics"]

    # Run all API calls concurrently
    responses = await asyncio.gather(
        *[call_api(ep) for ep in endpoints]
    )

    for resp in responses:
        print(resp)

asyncio.run(main())
```

Example 5: Timeout Handling

```
async def main():
    try:
        result = await asyncio.wait_for(
            call_api("slow_endpoint"),
            timeout=1.5
        )
        print("Success:", result)
    except asyncio.TimeoutError:
        print("Request timed out!")

asyncio.run(main())
```

Important Concepts & Best Practices

Concept	Description
<code>asyncio.run()</code>	Entry point to run async code (Python 3.7+)
<code>asyncio.create_task()</code>	Schedule coroutine to run concurrently
<code>asyncio.gather()</code>	Run multiple awaitables concurrently
<code>asyncio.sleep()</code>	Non-blocking sleep
<code>asyncio.wait_for()</code>	Add timeout to any awaitable
<code>asyncio.TaskGroup</code>	Modern way (Python 3.11+) for managing tasks

Best Practices:

- Never use `time.sleep()` inside async code → use `asyncio.sleep()`

- Always use `asyncio.run(main())` as the entry point
- Prefer `asyncio.gather()` or `TaskGroup` for running multiple tasks
- Be careful with blocking code — it can block the entire event loop